



COMSATS University Islamabad (CUI)

**Software Design Description
(SDS DOCUMENT)**

for

SkillEdge AI

Version 1.0

By

Farrukh Zubair Satti

CIIT/SP22-BAI-013/ISB

Raees Zakir

CIIT/SP22-BAI-039/ISB

Supervisor

Ms. Sanam Aamir

Co-Supervisor

Ms. Hilal Jan

Bachelor of Science in Computer Science (2022-2026)

Contents

1. Introduction.....	1
1.1 Purpose	1
1.2 Scope	1
1.3 Modules	2
1.3.1 Module 1: User Registration.....	2
1.3.2 Module 2: Interview Simulation	2
1.3.3 Module 3: Interview from Resume	3
1.3.4 Module 4: Evaluate Verbal/Non-Verbal Response	3
1.3.5 Module 5: Interview Highlights	3
1.3.6 Module 6: Overall Feedback.....	3
1.3.7 Module 7: Interactive Chatbot.....	4
1.3.8 Module 8: Progress Tracking and Analytics	4
2. Design Methodology and Software Process Model	4
2.1 Design Methodology:	4
2.2 Software Process Model	5
3. System Overview	5
3.1 Context Diagram	6
3.2 Box and Line Diagram	7
3.3 Architectural Design.....	8
4. Design Models.....	9
4.1 Activity Diagram	9
4.1.1 AD-01: Sign Up Process	9
4.1.2 AD-02: Sign In Process.....	10
4.1.3 AD-03: Forget Password Process:.....	11
4.1.4 AD-04: Profile Page	12
4.1.5 AD-05: Interview Type Selection	13
4.1.6 AD-06: Interview From Resume.....	14
4.1.7 AD-07: Question Answer Session.....	15
4.1.8 AD-08: Interview Evaluation	16
4.1.9 AD-09: Chat with Reports.....	17
4.1.10 AD-10: Chat with Resume	18
4.1.11 AD-11: Progress Tracking	19
4.2 Dataflow Diagrams.....	20

4.2.1	DFD Level 0.....	20
4.2.2	DFD Level 1.....	21
4.2.3	DFD Level 2-A	22
4.2.4	DFD Level 2-B.....	23
4.3	State Diagram	24
4.3.1	SD-01: User Registration	24
4.3.2	SD-02: Interview Simulation	24
4.3.3	SD-03: Interview From Resume	25
4.3.4	SD-04: AI Model Evaluation	25
4.3.5	SD-05: Chatbot.....	26
4.3.6	SD-06: Progress Tracking	26
5.	Data Design.....	27
5.1	Data Dictionary	27
5.1.1	User Profile Schemas	27
5.1.2	Interview Session Schema.....	28
5.1.3	Performance Report Schema.....	28
5.1.4	Resume Schema	29
5.1.5	Chatbot Interaction Schema	29
5.1.6	Progress Tracking Schema	30
6.	Implementation	30
6.1	Algorithm	30
6.1.1	Algorithm: Profile Management	30
6.1.2	Algorithm: Question Generation.....	32
6.1.3	Algorithm: Interview Simulation Flow	33
6.1.4	Algorithm: Answer Evaluation	34
6.1.5	Algorithm: Resume-Based Interview.....	36
6.1.6	Algorithm: Chatbot Query Handling.....	36
6.1.7	Algorithm: Progress Tracking Update	37
6.2	External APIs/SDKs.....	39
6.3	User Interface	40
6.3.1	Home Page	40
6.3.2	Profile Page	41
6.3.3	Select Interview Screen.....	42
6.3.4	Interview Simulation Screen	42
6.3.5	Interview Review Screen	43

7. Testing and Evaluation.....	44
7.1 Unit Testing:	44
7.1.1 Unit Testing 1: User Registration.....	44
7.1.2 Unit Testing 2: User Login.....	45
7.1.3 Unit Testing 3: Update Profile Page	45
7.1.4 Unit Testing 4: Question Generation & Session Management	46
7.2 Functional Testing:	47
7.2.1 Functional Testing 1: Interview Simulation Flow.....	47
7.2.2 Functional Testing 2: Test Cases for Resume-Based Interview.....	48
7.3 Integration Testing:	49
7.3.1 Integration Testing 1: Integration of User Registration and Profile Viewing	49
7.3.2 Integration Testing 2: User Registration (Login)	49
7.3.3 Integration Testing 3: Interview Simulator	50

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) *include the ones given at scope time both in doc and presentation	Action Taken
Revise use case diagram	We have discussed with the supervisor and revised the use case diagram.

Supervised by
Ms. Sanam Aamir

Signature _____

1. Introduction

SkillEdge AI is a web-based, AI-driven interview preparation platform that simulates realistic behavioral and technical interviews, analyzes both the content and delivery of user responses, and delivers detailed, actionable feedback. Built on a Next.js front end and FastAPI microservices, it leverages a fine-tuned Gemma3-1B model to generate domain-specific questions and advanced large-language models to evaluate answer accuracy, structure, and completeness. Beyond mock interviews, SkillEdge AI parses users' resumes to tailor questions, tracks performance trends over time through intuitive dashboards, and offers a context-aware chatbot that guides users on improving their verbal and non-verbal skills as well as optimizing their CVs, helping candidates build confidence and competence for real-world interviews.

1.1 Purpose

SkillEdge AI is a cutting-edge web-based tool that aims to transform interview preparation using AI-driven simulations and instant feedback. For students, job applicants, or professionals looking to sharpen their communication and technical skills, SkillEdge AI offers an in-depth and engaging learning experience.

The tool mimics real-life interview situations across various fields, leveraging sophisticated Natural Language Processing (NLP) and machine learning algorithms. From specialized Technical Interviews featuring AI-based answer analysis to Behavioral Assessments evaluating speech pronunciation, timing, and self-assurance, SkillEdge AI prepares users adequately for actual interview situations. Going beyond the limitations of traditional mock interviews, SkillEdge AI adds modules for Performance Reports with elaborate analytics, Interactive Chatbots offering individualized mentorship, and Progress Tracking that helps track improvements over time.

By providing an intelligent, adaptive, and user-focused method of interview training, SkillEdge AI enables users to gain confidence, improve their communication skills, and become job market competitive. The aim is to establish a dynamic, AI-based environment where users are provided with actionable insights and ongoing improvement opportunities, ultimately leading to career success and professional development.

1.2 Scope

The scope of SkillEdge AI is developing the AI-powered interview preparation platform, which would ensure users a lifelike, dynamic, and tailored mock interview experience. The platform would implement mock interview scenarios in any number of professional domains with domain specific questions pertinent to the profession that the user is interested in. Answers from the user will receive in-depth, AI-generated feedback, with two elements: first, verbal content, meaning answer relevance, correctness, structure, and completeness, and second, non verbal elements like voice clarity, pace, and filler words. It will also wrap up with a thorough performance report

detailing strengths and areas to improve upon and feature an interactive chatbot that steps the user through contextual advice on how to get better. The users of SkillEdge AI will also be able to track their progress over a few sessions, giving them a means of viewing improvement over time. Full access on the platform will be given via a very user-friendly web interface, so that navigation is seamless, and engagement promotes regular practice. In addition, SkillEdge AI will support various customizable interviewer personas to enable simulation of various interview styles and adaptive question based on user performance. The users can get their answers compared to ideal ones for more accurate learning. SkillEdge AI offers a rich combination of content-based and delivery-focused feedback and will be truly comprehensive and serve as an all-in-one tool for interview preparation. Users will be empowered to approach interviews confidently, clearly, and professionally.

1.3 Modules

1.3.1 Module 1: User Registration

The User Login and Onboarding module enables users to create accounts, log in, and provide essential information that personalizes their interview preparation experience. This initial setup captures details like the user's area of expertise, profession, and interview goals.

FE-1: New users sign up by providing their name, email, password, and information such as their area of expertise, preferred domains (e.g. technical, HR), desired roles, whether they are a student or professional, and their current career goals.

FE-2: Users log in using their username and password to access their personalized interview dashboard.

FE-3: Users can specify their interview goals, such as preparing for a particular type of job or skill, to tailor the interview experience based on their objectives.

1.3.2 Module 2: Interview Simulation

The Interview Simulation module will provide users with a real interview scenario, in which there will be an avatar (person) which will be asking questions, and the user will answer them.

FE-1: The system generates a set of domain-specific questions tailored to the user's expertise and experience level.

FE-2: An avatar asks questions from the user, simulating a real interview environment to create a more engaging experience.

FE-3: A certain time limit will be provided for complex questions, helping users practice thinking and responding quickly under pressure.

1.3.3 Module 3: Interview from Resume

The CV-Based Interview module allows candidates to upload their CVs, which the system then analyzes to pull out important details and then take the interview.

FE-1: Users will be able to upload their CV.

FE-2: System will extract important key points from cv like skills, education, experience etc.

FE-3: According to the extracted information, the system will ask questions from the user, questions will address his technical, behavioral and domain-specific knowledge and show him the corresponding results.

1.3.4 Module 4: Evaluate Verbal/Non-Verbal Response

The Non-Verbal Feedback module assesses various non-verbal aspects of communication, helping users improve delivery style and overall presence.

FE-1: User responses are evaluated by an LLM to check whether they are correct or not and then highlight answers which are not correct.

FE-1: The system analyzes the user's speech rate, providing feedback on whether they are speaking too quickly or too slowly.

FE-2: The system detects filler words (e.g., "um," "uh") and provides feedback on clarity and articulation.

FE-4: Feedback on tone and pitch is provided, helping users improve their vocal delivery to sound more confident and engaging.

FE-5: A score is generated for each answer, covering areas like relevance, depth, and alignment with the question, helping users understand their performance level. This score is used at the end to compute overall performance

1.3.5 Module 5: Interview Highlights

The Performance Report module generates a comprehensive report after each interview session, highlighting key metrics and offering targeted feedback.

FE-1: User can access 2 separate reports representing verbal and non-verbal responses.

FE-2: User can see his performance from details, graphs, bars provided in the reports.

FE-3: Performance is broken down into categories like technical knowledge and communication skills.

FE-4: At the end of each interview, there will be a detailed feedback section explaining why certain responses were strong or how could be improved.

1.3.6 Module 6: Overall Feedback

The Overall Feedback module evaluates the user's responses keeping in view the verbal and non-verbal feedback reports generated.

FE-1: Audio-derived non-verbal metrics (e.g., filler-word frequency, pause durations, speech rate, prosodic variation) are analyzed and correlated with verbal scores to detect patterns—such as rapid speech rate leading to incomplete explanations or use of excessive fillers words reducing clarity.

FE-2: The module highlights causal relationships between non-verbal behaviors and verbal outcomes. For instance, frequent filler words can interrupt idea flow, while rushed delivery may hinder concept articulation.

FE-3: Based on these interdependencies, the system generates prioritized action items and targeted exercises (e.g., paced storytelling drills, filler-reduction practices, speech rhythm modulation) that jointly address non-verbal and verbal performance improvements.

FE-4: The overall feedback report includes visual representations—such as line graphs, bar charts, histograms etc. and textual details describing all the relations.

1.3.7 Module 7: Interactive Chatbot

The Interactive Chatbot module provides personalized guidance by allowing users to engage in a follow-up discussion based on their performance report.

FE-1: Users can ask the chatbot questions about specific feedback points to gain a deeper understanding of their strengths and weaknesses.

FE-2: The chatbot suggests resources (e.g., articles, videos) and improvement tips tailored to the user's needs.

FE-3: Based on the user's interview goals, the chatbot recommends specific actions to help them progress toward those objectives.

1.3.8 Module 8: Progress Tracking and Analytics

The Progress Tracking and Analytics module provides users with insights into their improvement over time and helps them monitor their development across various skills.

FE-1: Users can review past performance reports to observe trends in their improvement over multiple sessions.

FE-2: The system shows trends in skills like communication, technical knowledge, and non-verbal communication, allowing users to understand which areas are improving.

FE-3: Users can set interview preparation goals and track their progress toward achieving them, based on metrics like improved scores and reduced filler word usage.

FE-4: Progress data is displayed on a user-friendly dashboard with visualizations like graphs and charts, making it easy to interpret development over time.

2. Design Methodology and Software Process Model

The following explains the design methodology and software process model for the SkillEdge AI.

2.1 Design Methodology:

The design methodology best suited for SkillEdge AI is Procedural Programming. In this approach, the application is organized as a series of well-defined functions, each responsible for a distinct task. For SkillEdge AI, functions will encapsulate core features such as:

- User Registration and Authentication
- Interview Simulation and Question Delivery

- Automated Verbal and Audio-based Non-verbal Evaluation
- Feedback Synthesis and Report Generation
- Progress Tracking and Analytics

By separating these concerns into discrete procedures, SkillEdge AI benefits from clearer code structure, simplified debugging, and targeted unit testing.

2.2 Software Process Model

The process model that will be followed for SkillEdge AI is the Incremental Model, as it allows the development of the system to be divided into independent and manageable modules. We will first implement 30% of the system, then move to 60%, and finally reach 100% functionality. With each increment, a working version of the software will be provided, allowing early testing, debugging, and feedback collection. This helps in improving the system progressively and makes it easier to implement changes or enhancements in the future. Therefore, the incremental model is the most appropriate choice for SkillEdge AI.

3. System Overview

SkillEdge AI is an intelligent, web-based platform that centralizes every stage of interview preparation into a cohesive, user-friendly environment. From user registration and profile management to realistic mock interviews—both behavioral and technical—the system automates question generation, response evaluation, and personalized feedback delivery. By parsing uploaded resumes, it tailors each interview to the candidate’s background and then provides separate, detailed verbal and non-verbal performance reports. An integrated, context-aware chatbot leverages those reports (and CV data) to guide users on improvement strategies, while a progress-tracking dashboard visualizes skill development trends over multiple sessions. Together, these modules streamline practice, feedback, and setting so that users can focus on honing their communication, technical knowledge, and confidence rather than administrative steps.

Under the hood, SkillEdge AI employs a microservices architecture: a Next.js frontend delivers fast, server-rendered pages and interactive React components, while FastAPI back-end services handle domain logic, data persistence, and LLM integrations. A fine-tuned Gemma3-1B model generates domain-specific questions, and additional large-language models evaluate answer content for correctness, structure, and completeness. Real-time audio analysis assesses speech rate, filler-word usage, tone, and pitch. All services communicate securely over HTTPS and WebSockets, ensuring low-latency interactions. This modular, scalable design not only guarantees consistent performance as the user base grows but also makes it easy to extend or replace individual components—such as swapping in new evaluation models or adding interview formats—based on ongoing user feedback and advances in AI.

3.1 Context Diagram

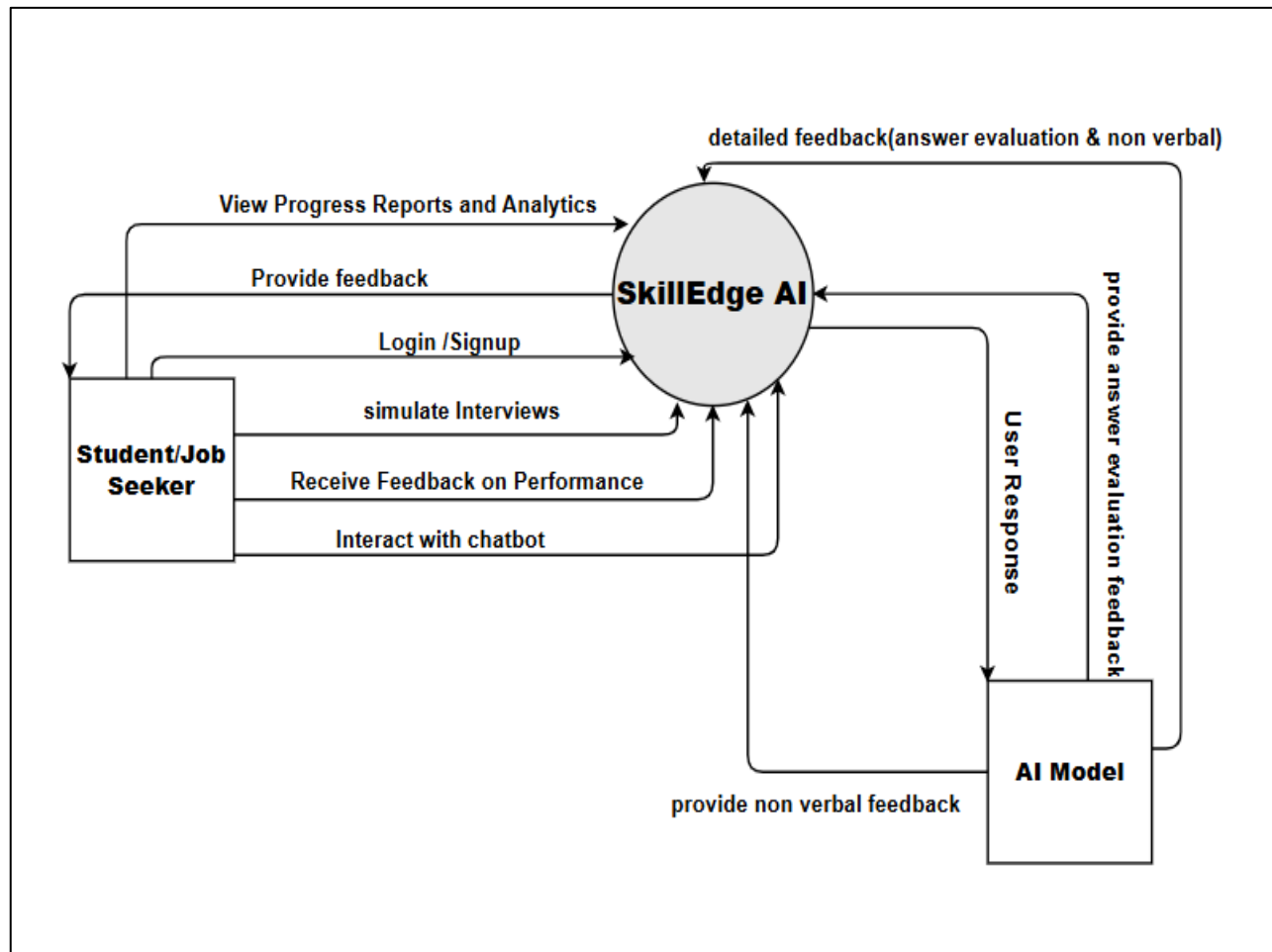


Figure 1: Context Diagram for SkillEdge AI

3.2 Box and Line Diagram

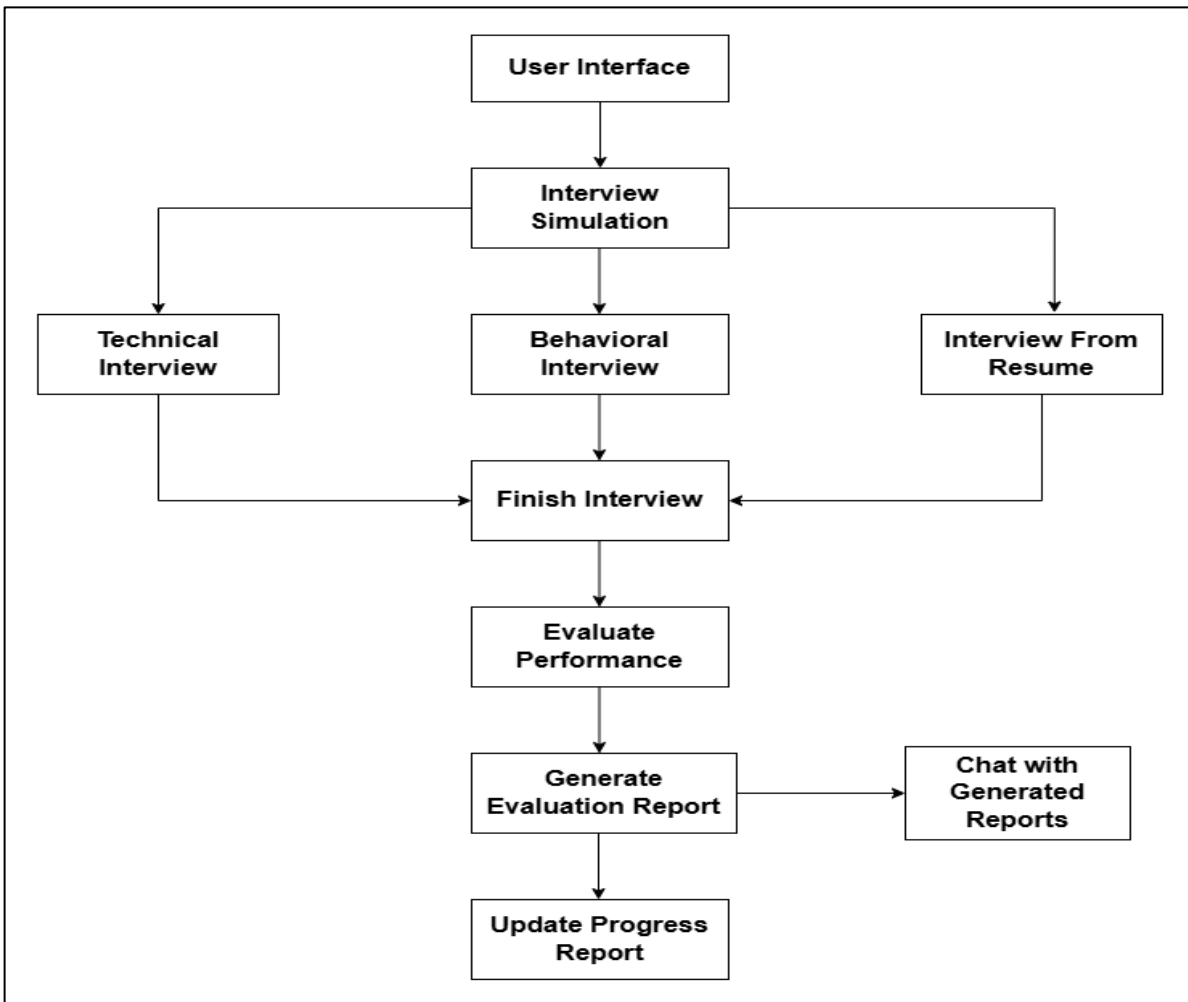


Figure 2: Box and Line Diagram for SkillEdge AI

The user interface begins with Interview Simulation, where users choose between Technical Interview (domain-specific questions) or Behavioral Interview (soft-skills assessment). After completing either session, the Finish Interview option concludes the simulation. Next, the system automatically Evaluates Performance by analyzing verbal/non-verbal responses. This triggers the Generate Evaluation Report function, producing detailed feedback with scores and improvement tips. Results are then logged via Update Progress Report for historical tracking. Alternatively, users can bypass standard simulations and initiate an Interview From Resume directly. Finally, at any post-evaluation stage, the Chat with Generated Reports feature allows interactive querying of feedback via an AI chatbot for personalized guidance.

3.3 Architectural Design

The SkillEdge AI platform is structured as a three-tiered system:

- **Presentation Tier:** A Next.js front end delivers fast, server-rendered React pages for user registration, interview simulation, report dashboards, and the chatbot interface.
- **Logic Tier:** FastAPI microservices implement core functionality—Gemma3-1B question generation, LLM-based answer evaluation, real-time audio analysis, context-aware chatbot, and progress-tracking services—each exposed via REST and WebSocket endpoints.
- **Data Tier:** A cloud-hosted Supabase database persists user profiles, interview sessions, parsed resume data, performance reports, and analytics metrics.

AI-driven microservices plug into the Logic Tier to automate question creation, feedback generation, and performance scoring, ensuring modularity, scalability, and easy extension.

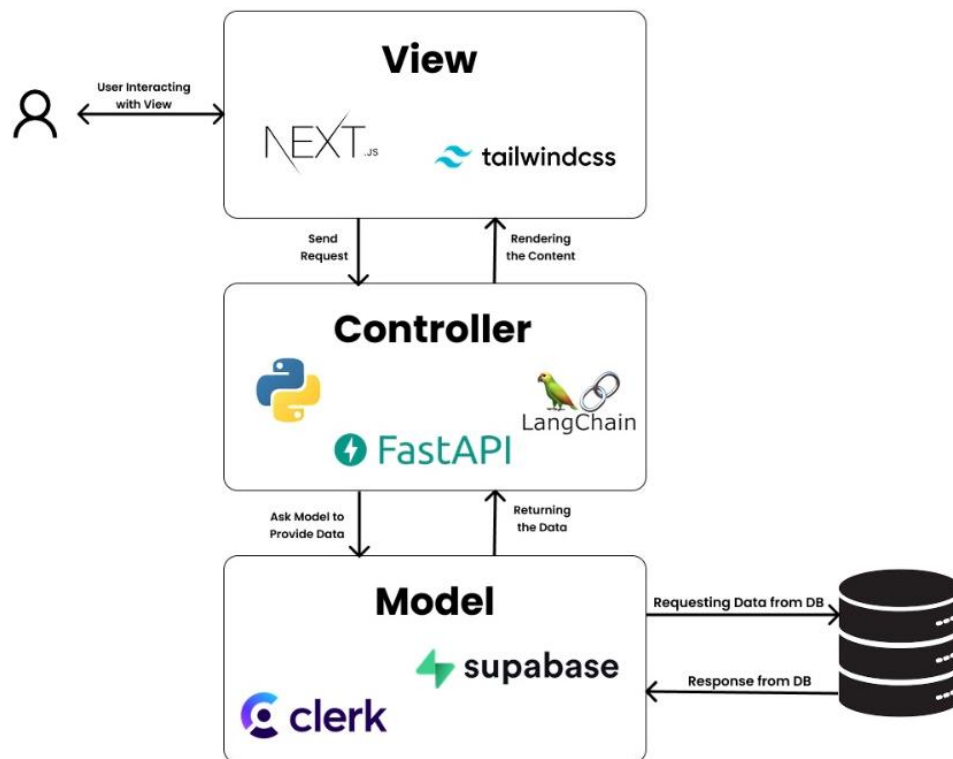


Figure 3: MVC Architecture Diagram for SkillEdge AI

4. Design Models

4.1 Activity Diagram

4.1.1 AD-01: Sign Up Process

The sign-up activity diagram depicts the process of new user registration. It outlines how a user provides the required information, such as email and password, which is then validated and stored in the system to create a new account

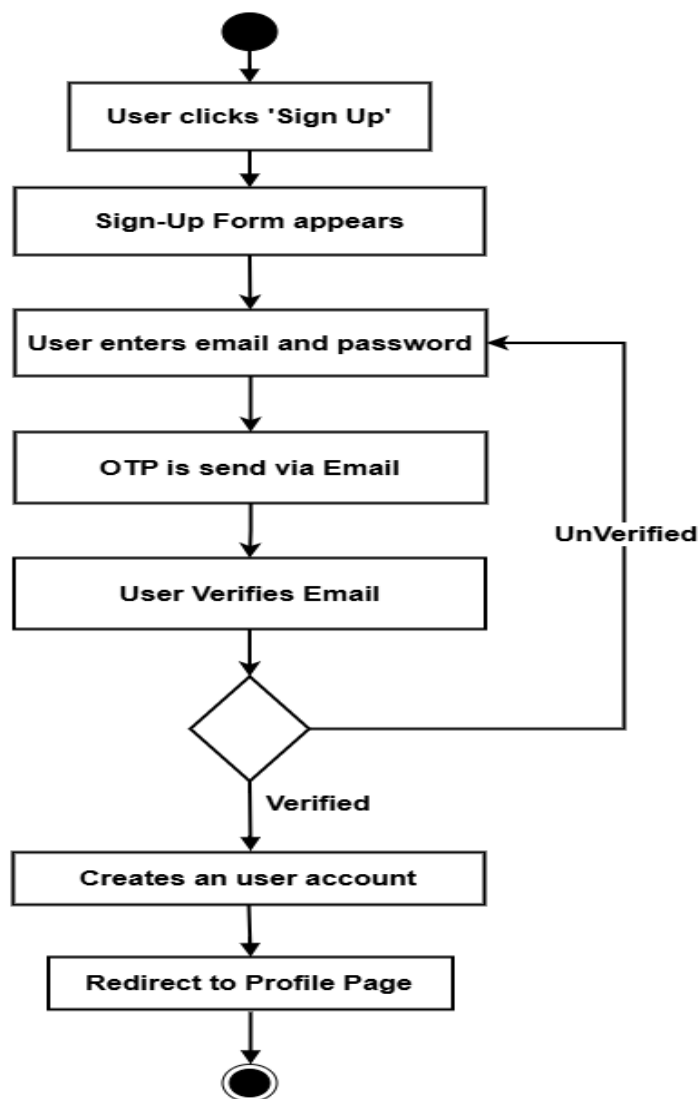


Figure 4: Activity Diagram for User Signing Up

4.1.2 AD-02: Sign In Process

The sign-in activity diagram illustrates the process a user follows to log into the system using valid credentials.

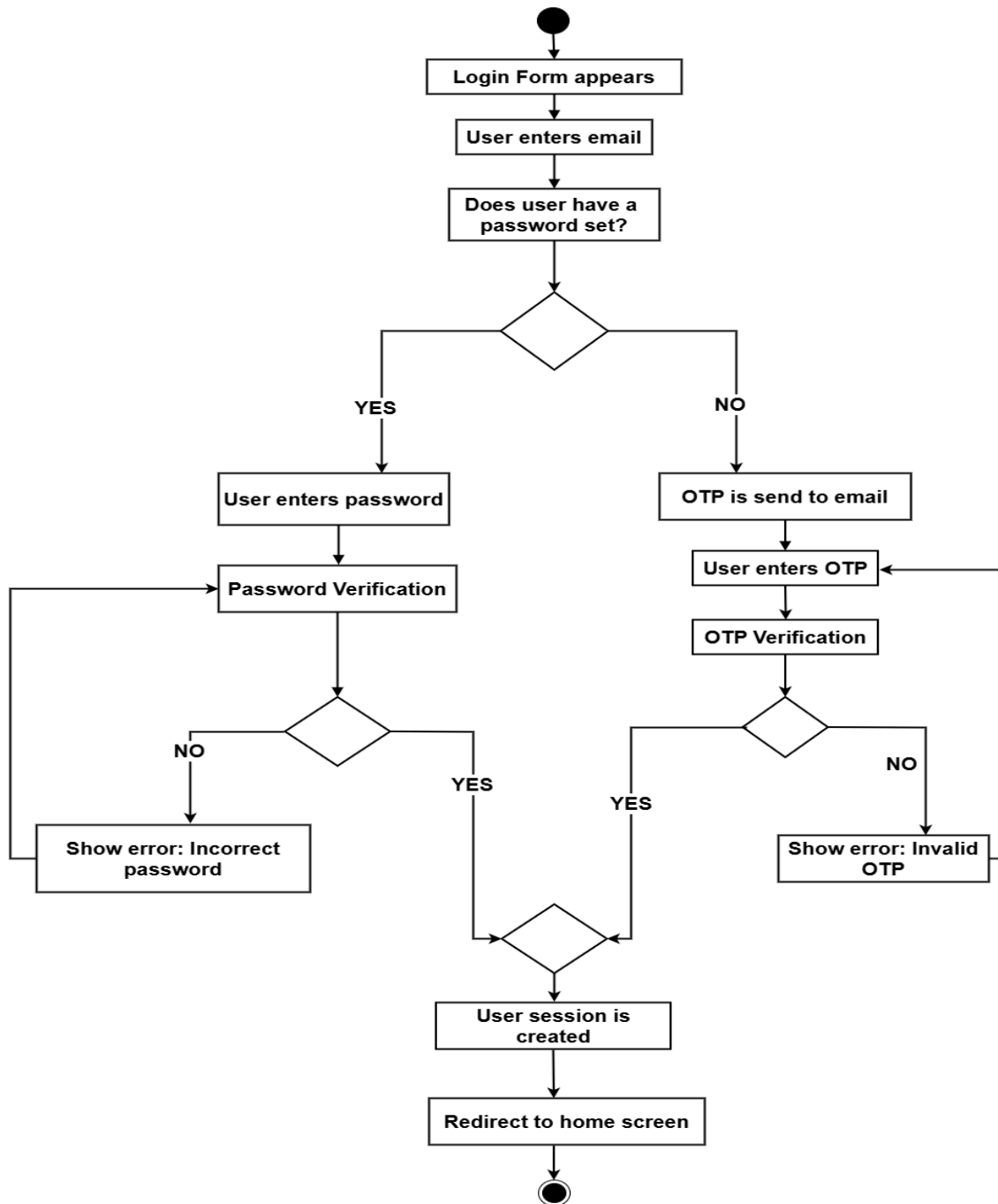


Figure 5: Activity Diagram for User Signing In

4.1.3 AD-03: Forget Password Process:

The activity diagram shows how a user can reset his forgotten password using his email address.

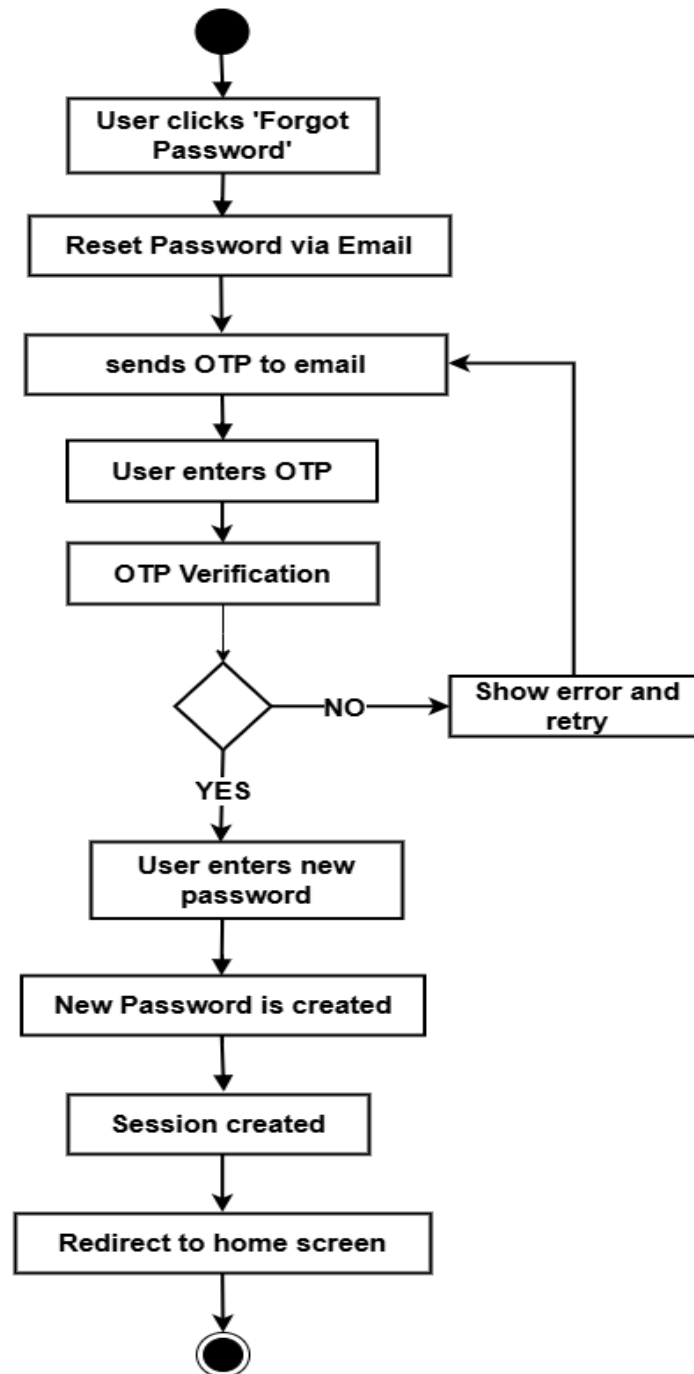


Figure 6: Activity Diagram for Idea Validation

4.1.4 AD-04: Profile Page

This activity diagram shows how the user can update his profile information.

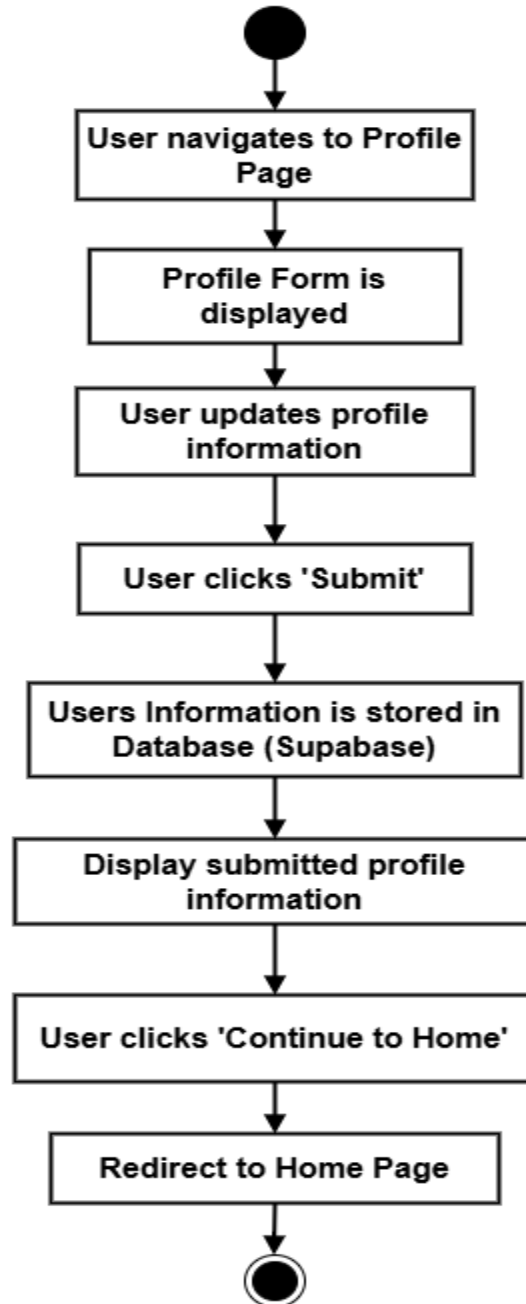


Figure 7: Activity Diagram for Supervisor Student Matching

4.1.5 AD-05: Interview Type Selection

The Interview Simulation activity diagrams shows how the user will start the interview and select the type of interview (technical or behavioral) etc.

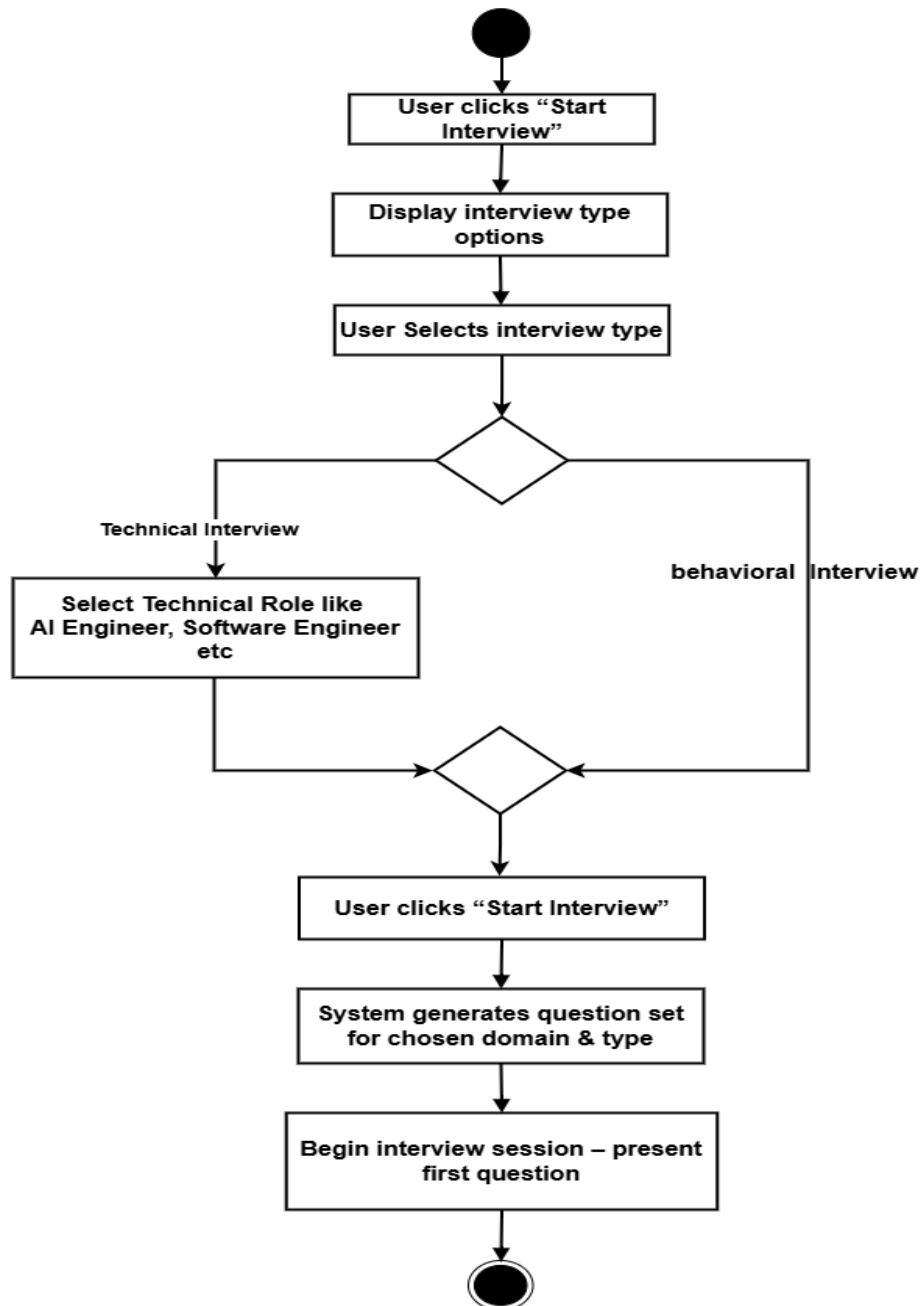


Figure 8: Activity Diagram for AI and Plagiarism Report Checking

4.1.6 AD-06: Interview From Resume

This activity diagram shows if the user wants to give an interview specifically from his resume, he will first need to upload his resume in database and then start the interview.

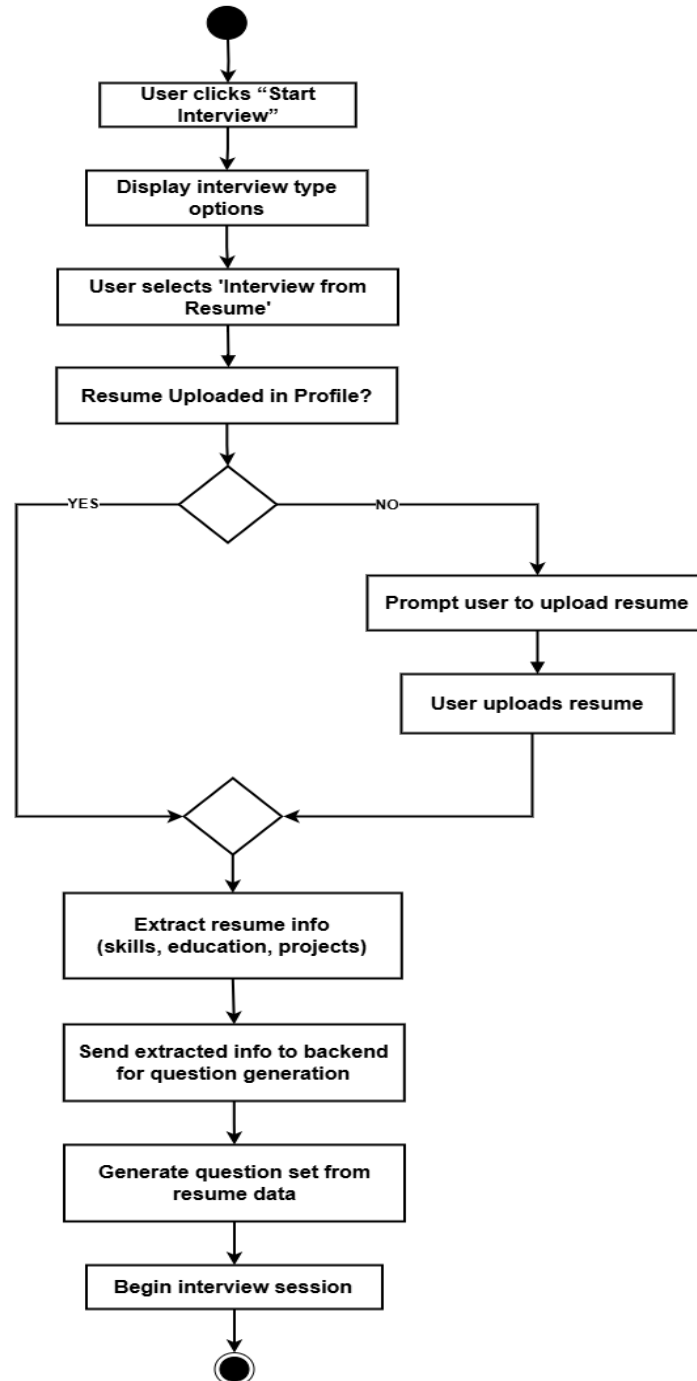


Figure 9: Activity Diagram for Feedback and Grading

4.1.7 AD-07: Question Answer Session

The Question Answer Session activity diagram shows the flow when the user starts the interview, answers the question, goes to next question and at last finish the interview and then decide to give interview again or communicate with chatbot on his generated reports reflecting his performance.

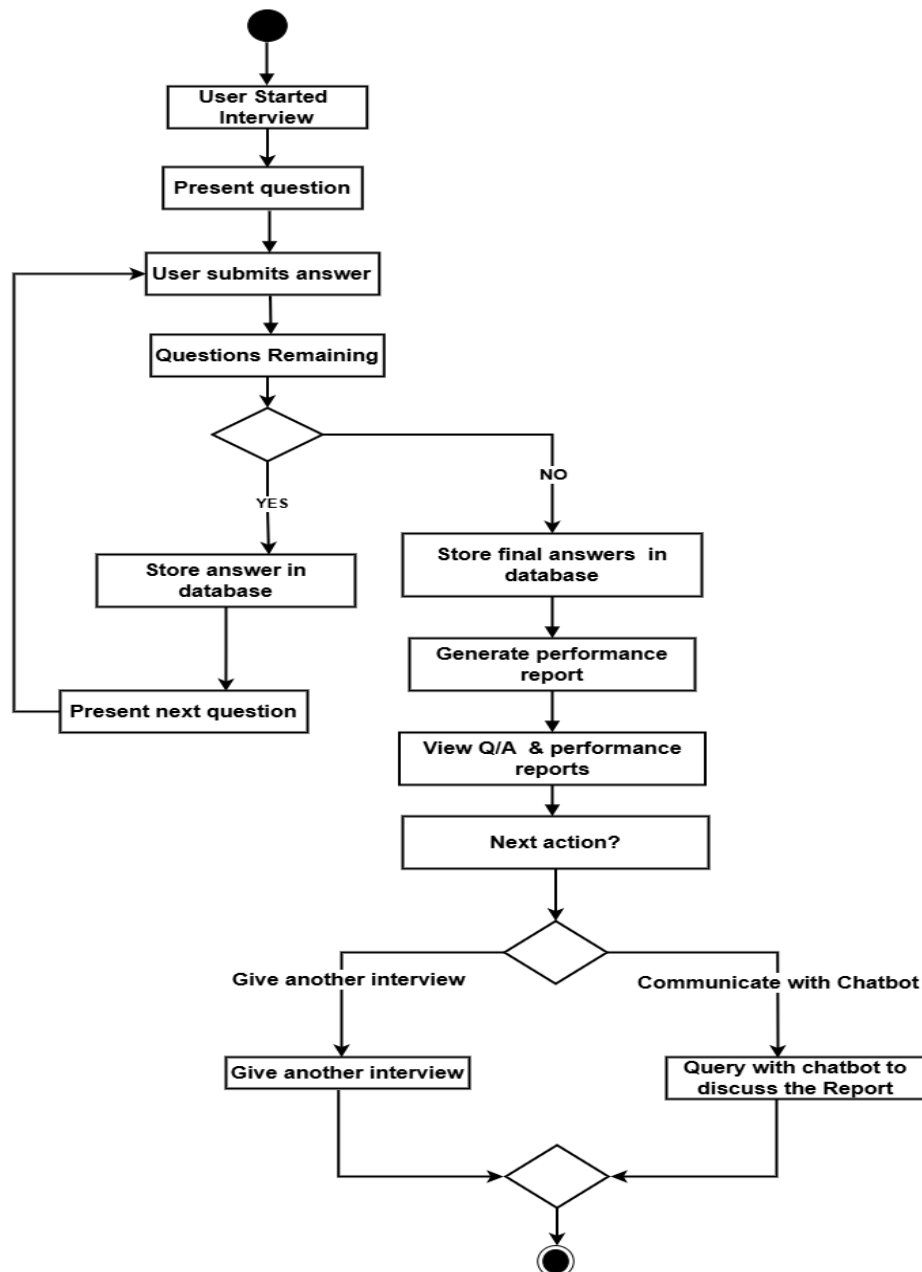


Figure 10: Activity Diagram for Alumni Projects and Networking

4.1.8 AD-08: Interview Evaluation

The Interview Evaluation activity diagram shows that when the user finishes the interview, all the questions and answers are stored in the database, then on the basis of user performance, verbal and non-verbal reports are generated.

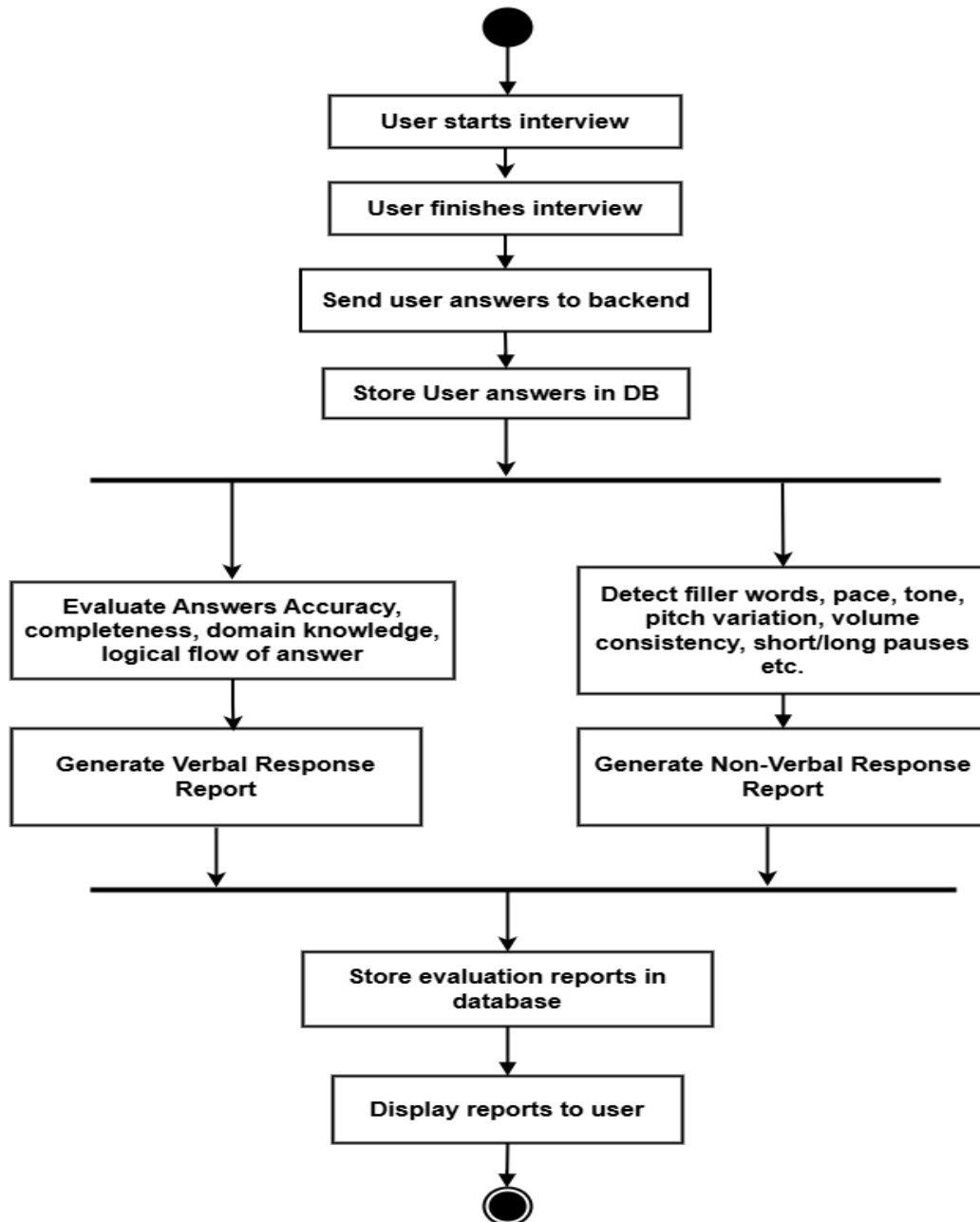


Figure 11: Activity Diagram for Answer Evaluation

4.1.9 AD-09: Chat with Reports

This activity diagram shows that when the performance reports are generated, user can access chatbot and query it and get back a response in context of the generated reports.

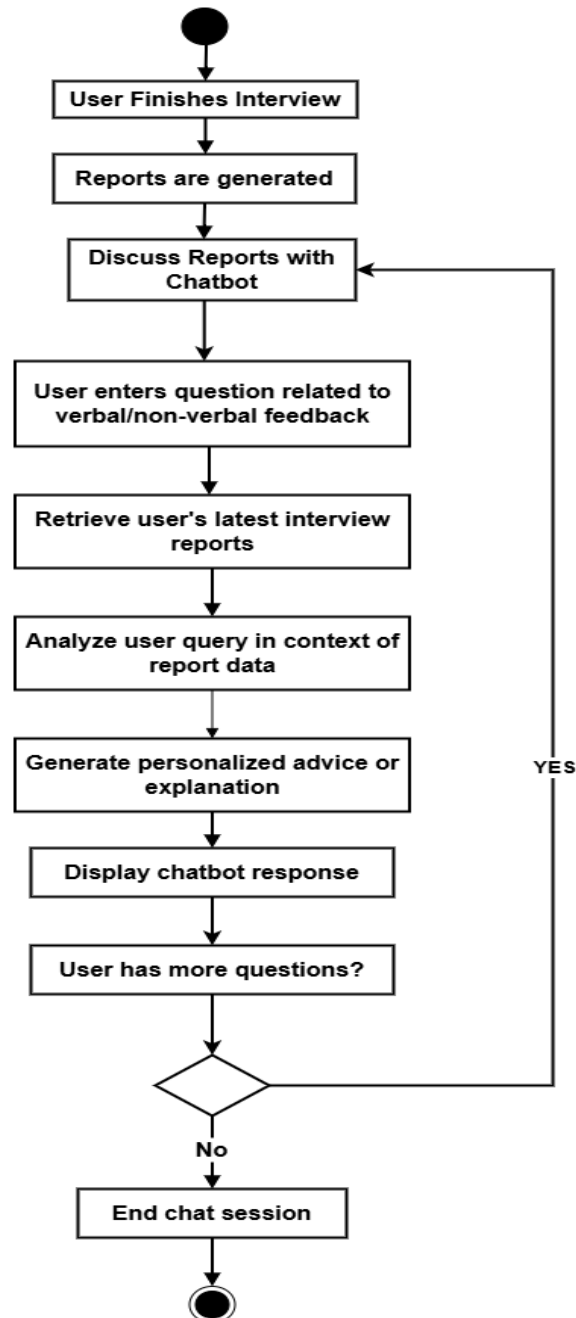


Figure 12: Activity Diagram for Chat with Reports

4.1.10 AD-10: Chat with Resume

This activity diagram shows that if the user wants to query about his resume, selects chat with resume, then have to upload his resume and query the chatbot.

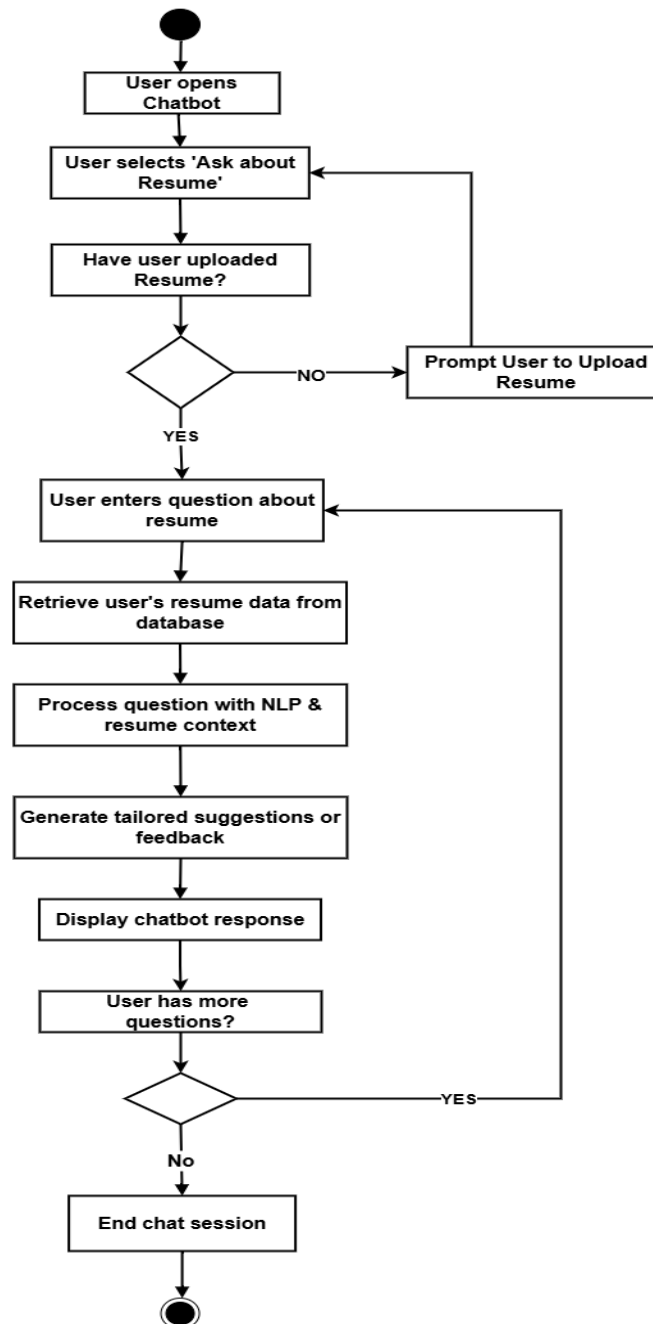


Figure 13: Activity Diagram for Chat with Resume

4.1.11 AD-11: Progress Tracking

This activity diagram shows how user can see his progress and improvement by accessing the Progress Tracking module.

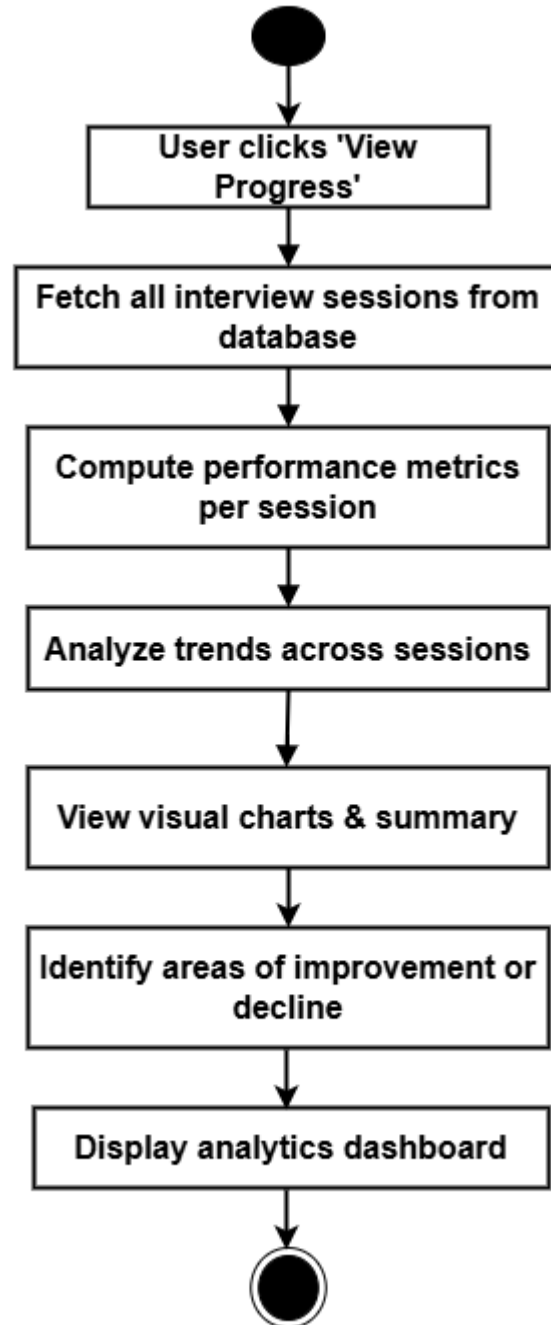


Figure 14: Activity Diagram for Progress Tracking

4.2 Dataflow Diagrams

Below are the dataflow diagrams of SkillEdge AI.

4.2.1 DFD Level 0

Below figure shows the DFD Level 0 diagram of SkillEdge AI.

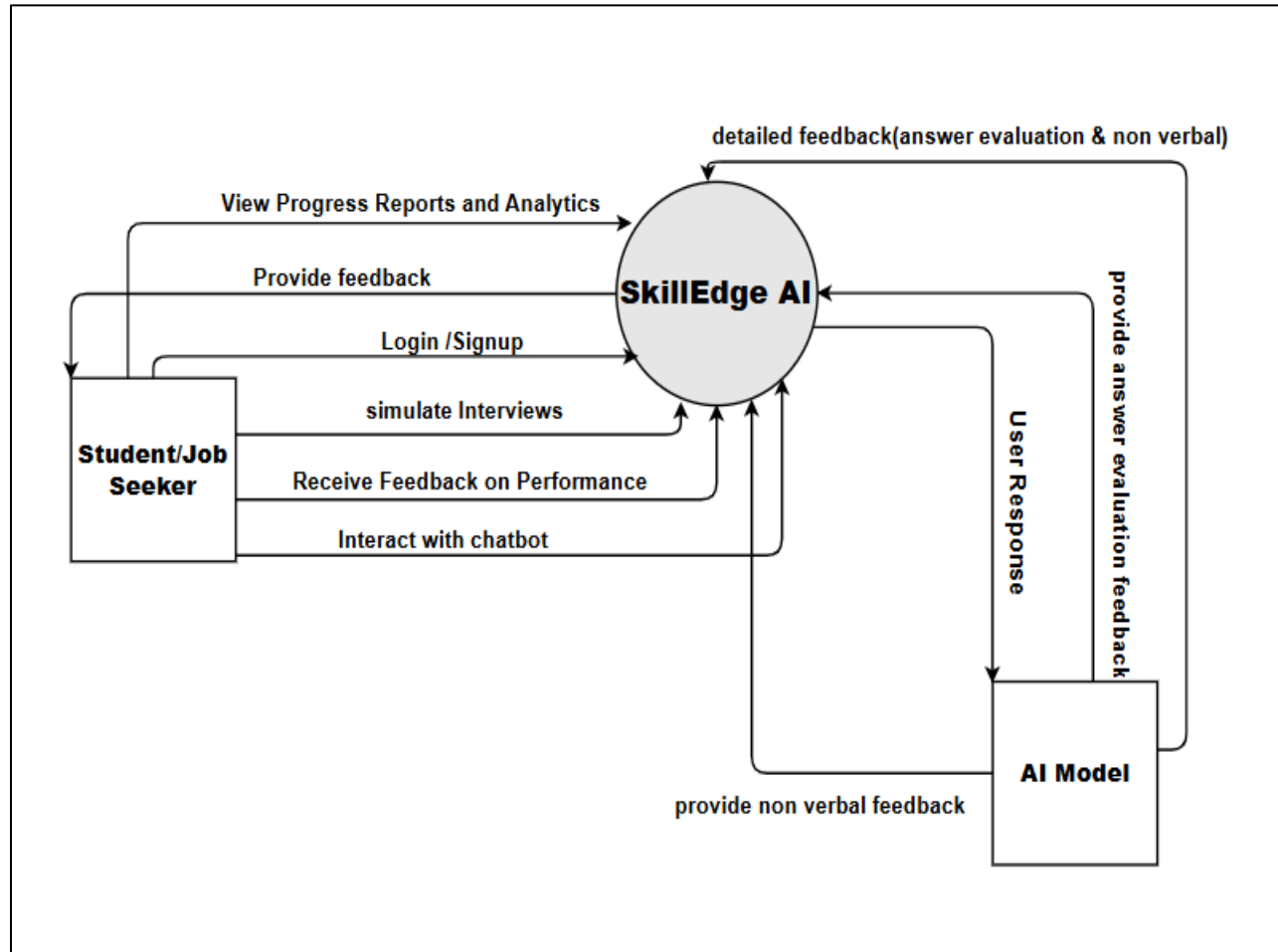


Figure 15: Context Diagram for SkillEdge AI.

4.2.2 DFD Level 1

Below figure shows the DFD Level 1 diagram for SkillEdge AI.

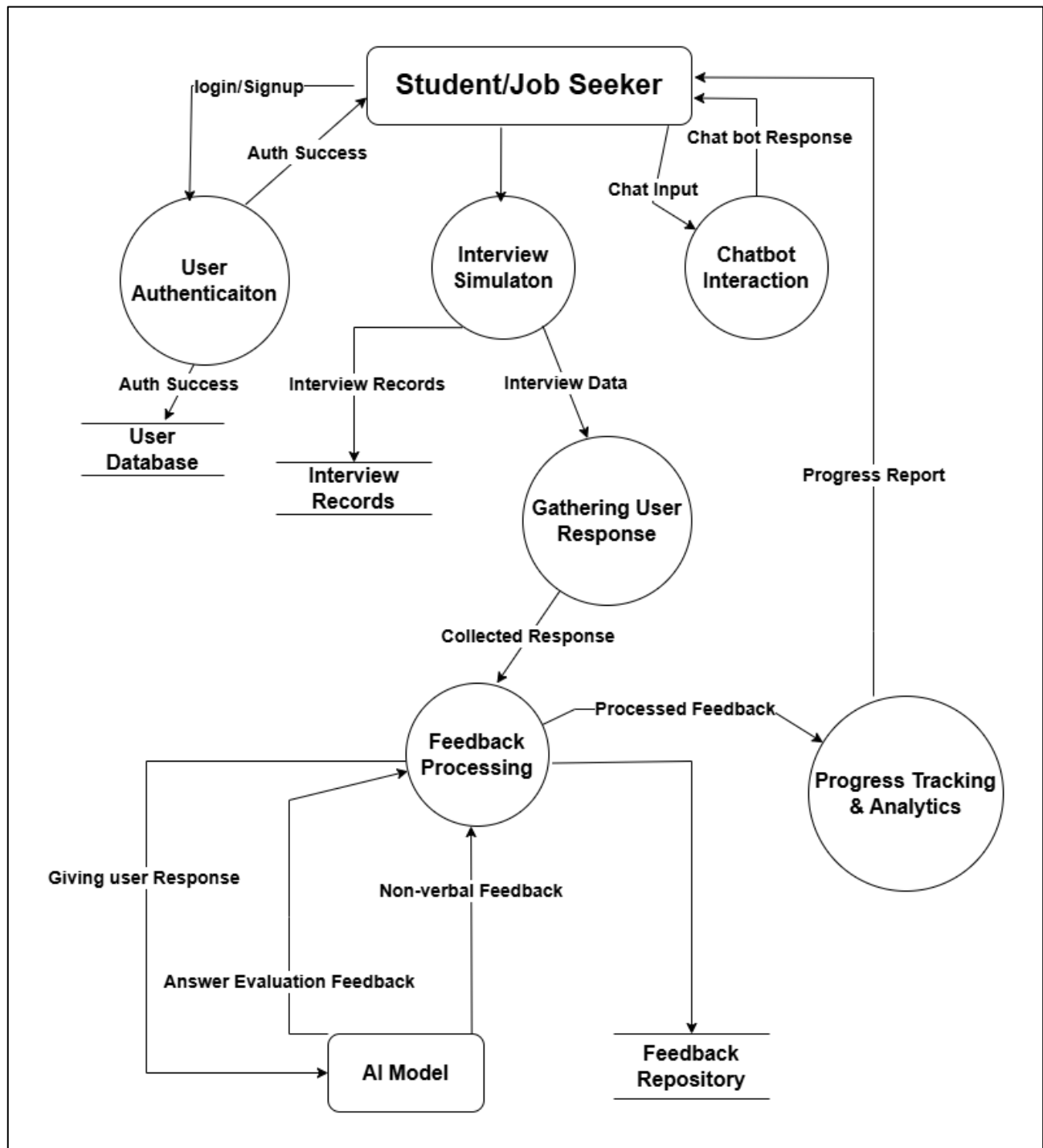


Figure 16: Sequence Diagram for DFD Level 1

4.2.3 DFD Level 2-A

Below figure show the DFD Level 2-A diagram of SkillEdge AI.

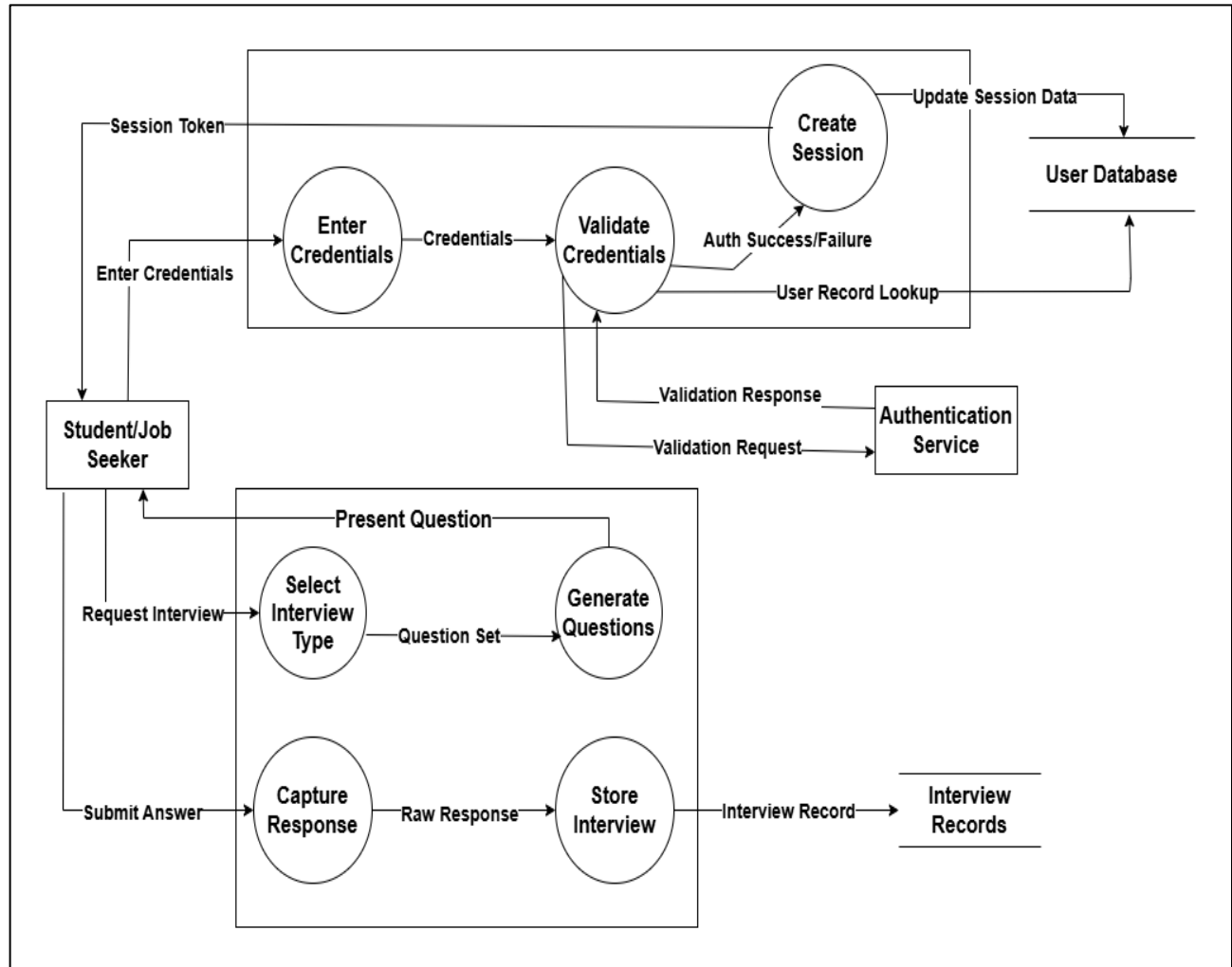


Figure 17: Sequence Diagram for DFD Level 2-A

4.2.4 DFD Level 2-B

Below figure show the DFD Level 2-B diagram of SkillEdge AI.

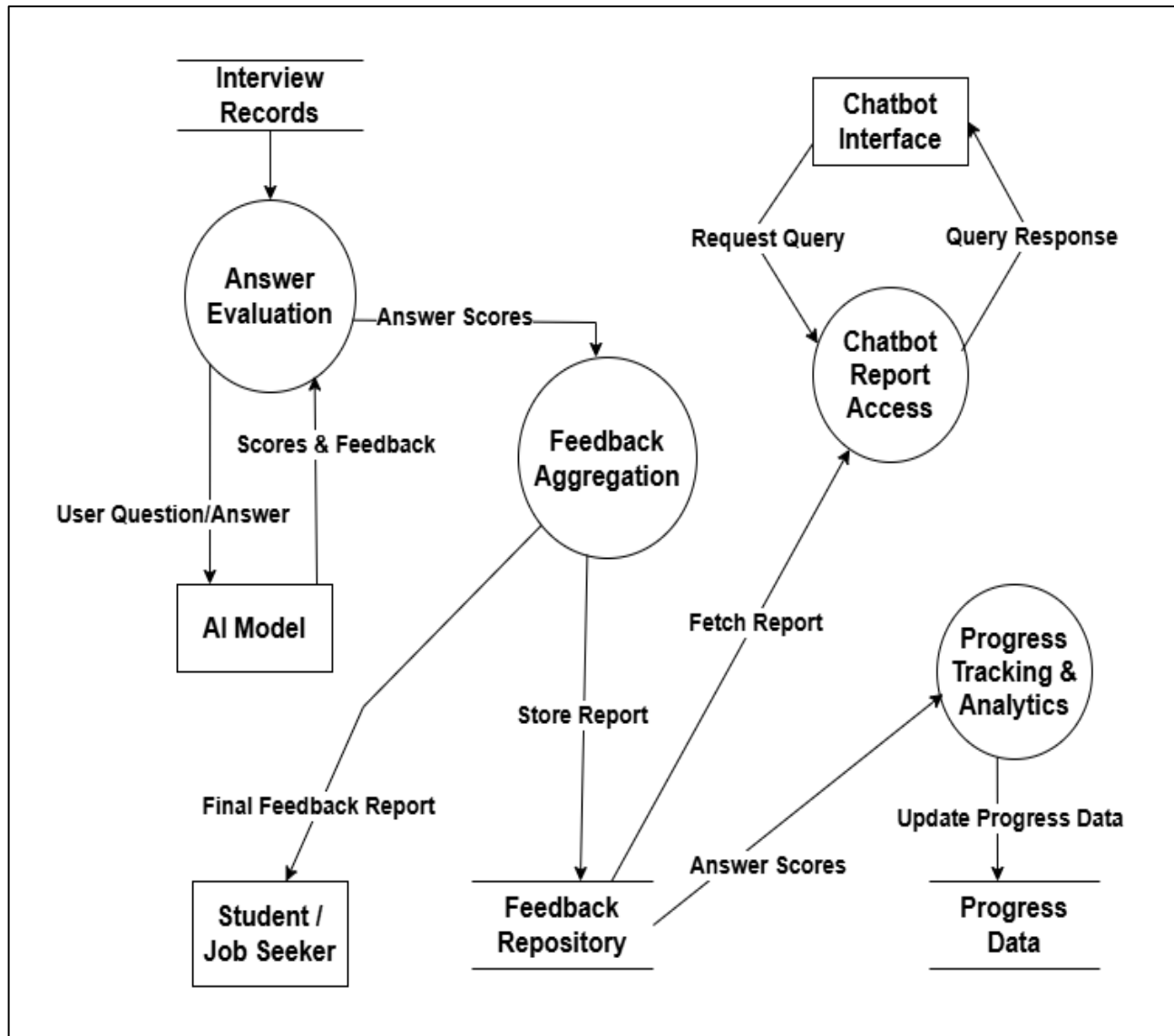


Figure 18: Sequence Diagram for DFD Level 2-B

4.3 State Diagram

Below are the state machine diagrams for SkillEdge AI.

4.3.1 SD-01: User Registration

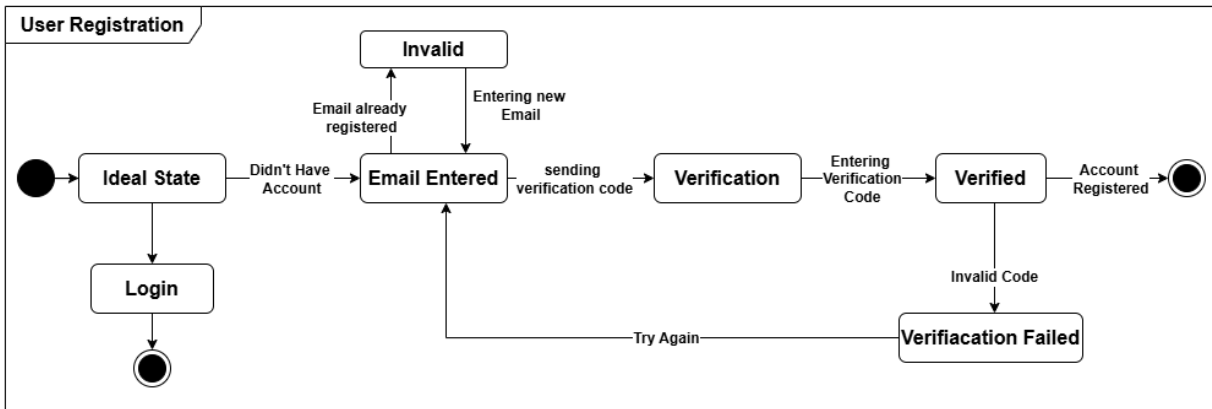


Figure 19: State Machine Diagram for User Registration

4.3.2 SD-02: Interview Simulation

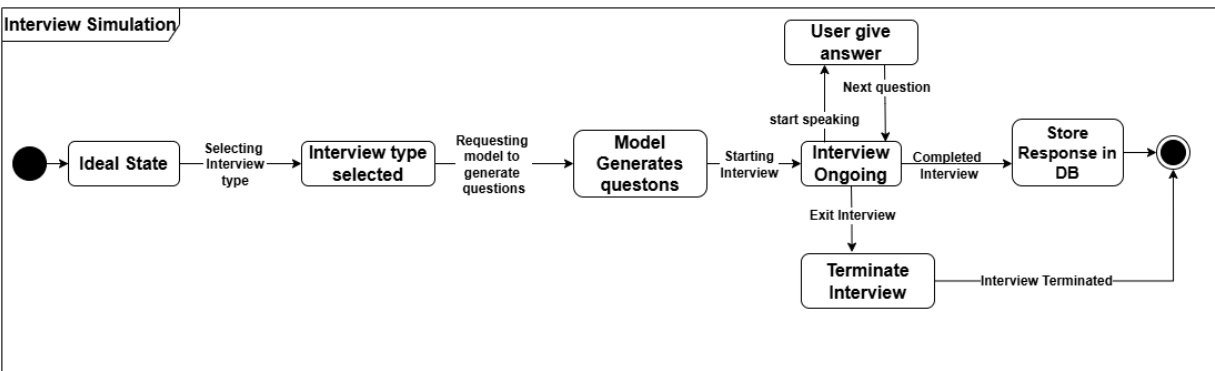


Figure 20: State Machine Diagram for Interview Simulation

4.3.3 SD-03: Interview From Resume

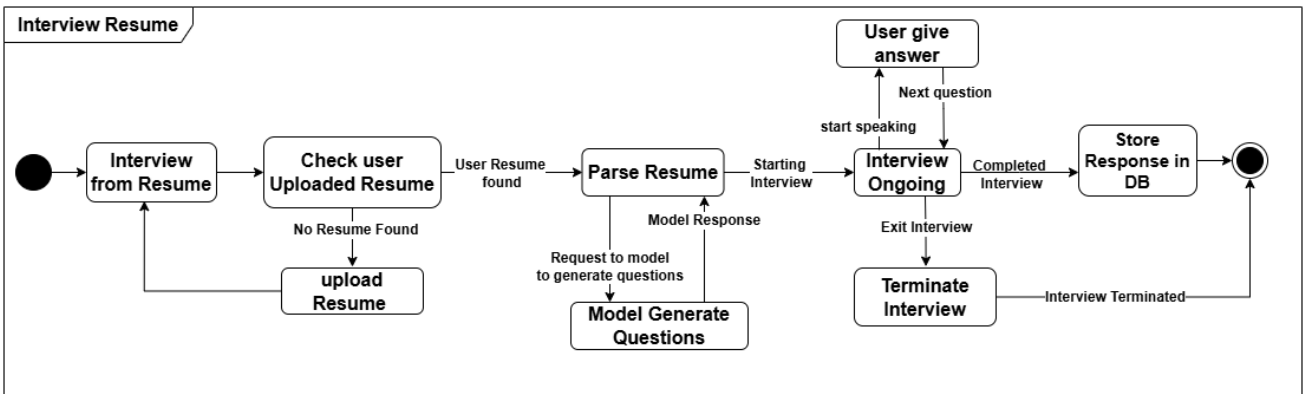


Figure 21: State Machine Diagram for Interview from Resume

4.3.4 SD-04: AI Model Evaluation

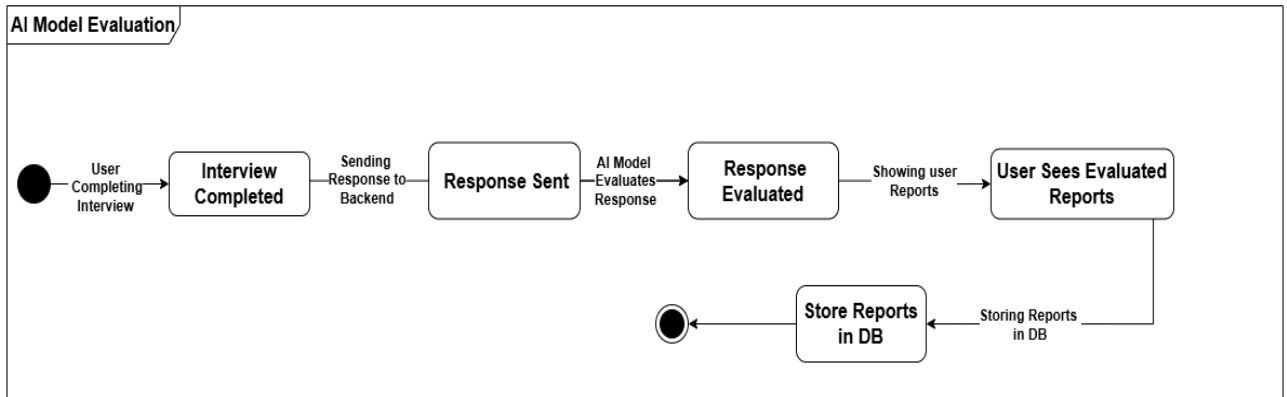


Figure 22: State Machine Diagram for User Registration

4.3.5 SD-05: Chatbot

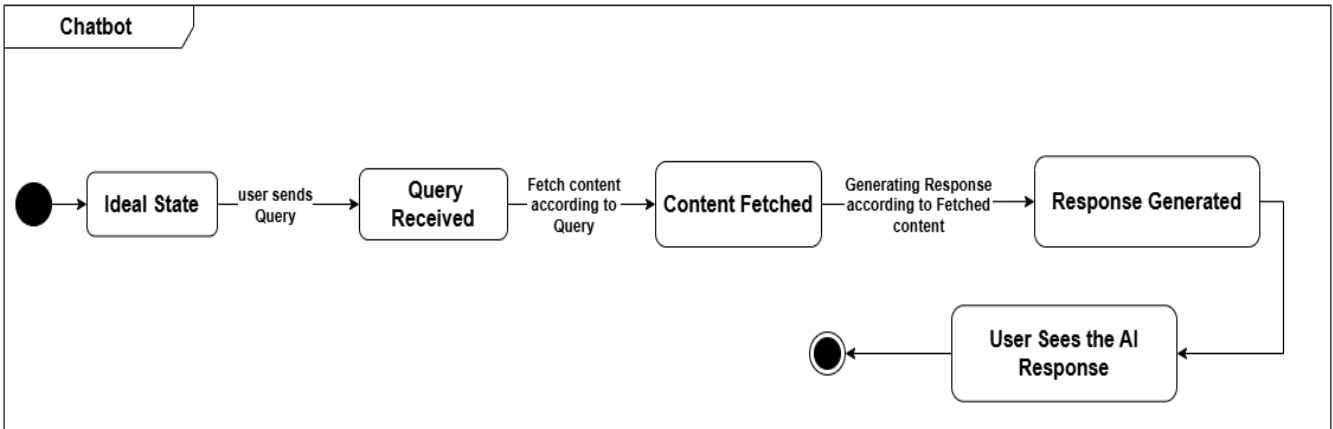


Figure 23: State Machine Diagram for Chatbot

4.3.6 SD-06: Progress Tracking

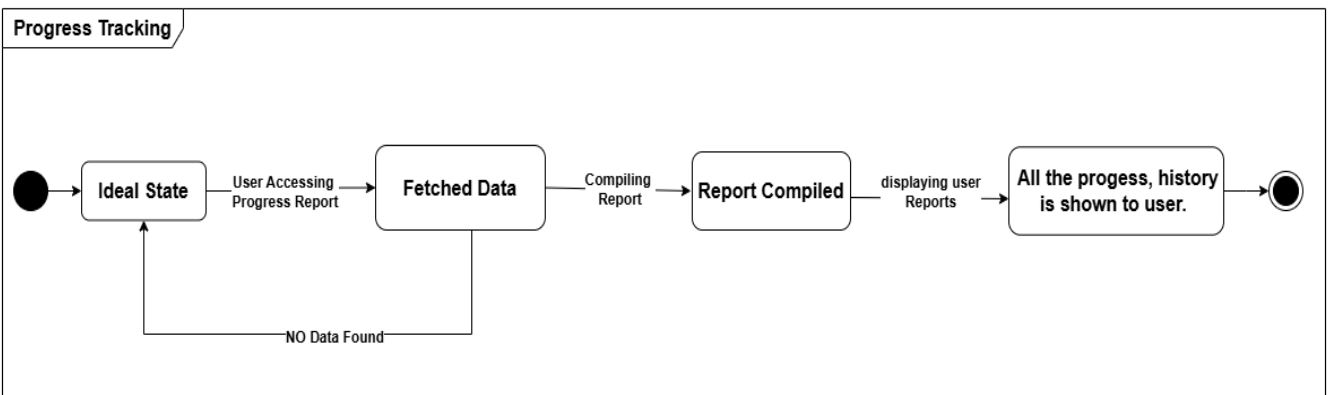


Figure 24: State Machine Diagram for Chatbot

5. Data Design

The data for SkillEdge AI is stored in Supabase database, a non-relational database well-suited for handling diverse and evolving data structures. All major data entities are represented through JSON-based schemas and accessed using an asynchronous Supabase client integrated with the FastAPI backend. Each schema corresponds to a core component or feature within the system, such as user profiles, interview sessions, performance reports, and chatbot interactions. The following section describes these data models and their structure through a detailed data dictionary.

5.1 Data Dictionary

5.1.1 User Profile Schemas

```
{
  "profile_id": "string",
  "user_id": "string",
  "first_name": "string",
  "last_name": "string",
  "skills": ["string"],
  "industry": "string",
  "position": "string",
  "experience": "integer",
  "bio": "string",
  "resume_url": "string",
  "created_at": "datetime",
  "updated_at": "datetime"
}
```

Table 5.1.1: Data Dictionary for User Profile

Field	Data Type	Description
profile_id	string	Unique identifier for the profile record.
user_id	string	Foreign key linking to the User (user_id in User schema).
first_name	string	User's first name (from the form).
last_name	string	User's last name.
skills	list	Array of skills (parsed from the comma-separated input).
industry	string	Selected industry.
position	string	Selected position/role.
experience	integer	Years of experience (parsed from the numeric input).
bio	string	User's professional bio or summary.
resume_url	string	Public URL to the uploaded résumé PDF (if any).
created_at	datetime	Timestamp when the profile was first created.
updated_at	datetime	Timestamp of the last profile update (PUT to /api/profile).

5.1.2 Interview Session Schema

```
{
  "session_id": "string",
  "user_id": "string",
  "interview_type": "string",
  "questions": ["string"],
  "responses": ["string"],
  "start_time": "datetime",
  "end_time": "datetime"
}
```

Table 5.1.2: Data Dictionary for Interview Session Schema

Field	Data Type	Description
session_id	string	Unique identifier for the interview session.
user_id	string	ID of the user giving the interview.
interview_type	string	Type of interview (Technical/Behavioral).
questions	list	List of questions asked during the session.
responses	list	List of user responses.
start_time	datetime	Start time of the session.
end_time	datetime	End time of the session.

5.1.3 Performance Report Schema

```
{
  "report_id": "string",
  "session_id": "string",
  "verbal_score": "float",
  "nonverbal_score": "float",
  "feedback_summary": "string",
  "report_generated_at": "datetime"
}
```

Table 5.1.3: Data Dictionary for Report Schema

Field	Data Type	Description
report_id	string	Unique identifier for the performance report.
session_id	string	Associated interview session.
verbal_score	float	Score evaluating content accuracy and structure.
nonverbal_score	float	Score evaluating speech rate, pitch, tone, etc.
feedback_summary	string	Summary of strengths and areas for improvement.
report_generated_at	datetime	When the report was generated.

5.1.4 Resume Schema

```
{
  "resume_id": "string",
  "user_id": "string",
  "filename": "string",
  "parsed_data": {
    "skills": ["string"],
    "experience": ["string"],
    "education": ["string"]
  },
  "uploaded_at": "datetime"
}
```

Table 5.1.4: Data Dictionary for Resume Schema

Field	Data Type	Description
resume_id	string	Unique identifier for uploaded resume.
user_id	string	Associated user ID.
filename	string	Name of the uploaded file.
parsed_data	object	Extracted fields like skills, projects, etc.
uploaded_at	datetime	Timestamp of resume upload.

5.1.5 Chatbot Interaction Schema

```
{
  "chat_id": "string",
  "user_id": "string",
  "query_type": "string",
  "query": "string",
  "response": "string",
  "timestamp": "datetime"
}
```

Table 5.1.5: Data Dictionary for Chatbot Interaction Schema

Field	Data Type	Description
chat_id	string	Unique ID for each chatbot interaction.
user_id	string	Associated user who initiated the query.
query_type	string	Type of query (Report-based / Resume-based).
query	string	User's question or request.
response	string	Chatbot's AI-generated reply.
timestamp	datetime	Time when the query was made.

5.1.6 Progress Tracking Schema

```
{
  "progress_id": "string",
  "user_id": "string",
  "session_ids": ["string"],
  "trend_data": {
    "verbal": ["float"],
    "nonverbal": ["float"]
  },
  "last_updated": "datetime"
}
```

Table 5.1.6: Data Dictionary for Progress Tracking Schema

Field	Data Type	Description
progress_id	string	Unique identifier for progress tracking record.
user_id	string	User whose progress is being tracked.
session_ids	list	List of associated session IDs.
trend_data	object	Trend scores over time for verbal and non-verbal metrics.
last_updated	datetime	Last time the progress data was updated.

6. Implementation

The following are some main algorithms used in the system.

6.1 Algorithm

6.1.1 Algorithm: Profile Management

Table 6.1.1 Profile Management

Algorithm 1: User Registration
Input: name, email, password
Output: success or error message
<pre>1: Begin 2: validate(name, email, password) 3: if any validation fails then 4: return error("Invalid input") 5: end if 6: pwd_hash ← hash(password) 7: user ← { name, email, password_hash: pwd_hash } 8: result ← db.users.insert_one(user) 9: if result.acknowledged then 10: return success("User registered successfully") 11: else 12: return error("Database error") 13: end if 14: End</pre>

Algorithm 2 User Login**Input: email, password****Output: JWT access token or error message**

```

1: Begin
2:  user ← db.users.find_one({ email })
3:  if user is None then
4:    return error("Invalid email or password")
5:  end if
6:  if not verify(password, user.password_hash) then
7:    return error("Invalid email or password")
8:  end if
9:  access_token ← generate_jwt({ user_id: user._id }, expiry)
10: refresh_token ← generate_jwt({ user_id: user._id }, refresh_expiry)
11: return { access_token, refresh_token }
12: End

```

Algorithm 3 Token Refresh**Input: refresh_token****Output: new access token or error message**

```

1: Begin
2:  payload ← verify_jwt(refresh_token)
3:  if payload invalid or expired then
4:    return error("Invalid or expired refresh token")
5:  end if
6:  new_token ← generate_jwt({ user_id: payload.user_id }, expiry)
7:  return { access_token: new_token }
8: End

```

Algorithm 4 Password Reset Request**Input: email****Output: success or error message**

```

1: Begin
2:  user ← db.users.find_one({ email })
3:  if user is None then
4:    return error("Email not registered")
5:  end if
6:  reset_token ← generate_secure_token()
7:  expires_at ← now() + RESET_TOKEN_TTL
8:  db.password_resets.insert_one({
    user_id: user._id, reset_token, expires_at
  })
9:  send_email(email, reset_token_link(reset_token))
10: return success("Password reset email sent")
11: End

```

Algorithm 5 Password Update
Input: reset_token, new_password
Output: success or error message
<pre> 1: Begin 2: record ← db.password_resets.find_one({ reset_token }) 3: if record is None or record.expires_at < now() then 4: return error("Invalid or expired token") 5: end if 6: new_hash ← hash(new_password) 7: result ← db.users.update_one({ _id: record.user_id }, { \$set: { password_hash: new_hash } }) 8: if result.modified_count == 1 then 9: db.password_resets.delete_many({ user_id: record.user_id }) 10: return success("Password updated successfully") 11: else 12: return error("Database error") 13: end if 14: End </pre>

6.1.2 Algorithm: Question Generation

Table 6.1.2 Question Generation

Algorithm 1 Question Generation.
Input: user_id, interview_type, topic_preferences
Output: questions (List of strings)
<pre> 1: Begin 2: profile ← db.users.find_one({ _id: user_id }) 3: context ← { type: interview_type, topics: topic_preferences, experience: profile.experience_level } 4: prompt ← build_question_prompt(context) 5: response ← call_llm("gemma3-1b-finetuned", prompt, max_tokens=500) 6: raw_questions ← parse_questions(response) 7: questions ← remove_duplicates(raw_questions) 8: if questions is empty then 9: questions ← default_question_set(interview_type) 10: end if 11: Return questions 12: End </pre>

Algorithm 2 constructQuestionPrompt.
Input: context
Output: prompt (String)
<pre> 1: Begin 2: baseText ← "Generate a set of " + context.type + " interview questions" 3: if context.topics not empty then 4: baseText ← baseText + " on " + join(context.topics, ", ") 5: end if 6: baseText ← baseText + ". Experience level: " + context.experienceLevel 7: return baseText 8: End </pre>

6.1.3 Algorithm: Interview Simulation Flow

Table 6.1.3 Interview Simulation Flow

Algorithm 1: Interview Simulation Flow
Input: user_id, interview_type
Output: session_id (string)
<pre> 1: Begin 2: questions ← generate_questions(user_id, interview_type) 3: session_id ← generate_uuid() 4: init_session(session_id, user_id, interview_type, questions) 5: for each q in questions do 6: send_to_frontend(session_id, q) 7: resp ← await_frontend_response(session_id, q) 8: record_response(session_id, q, resp) 9: end for 10: close_session(session_id) 11: Return session_id 12: End </pre>

Algorithm 2: awaitClientResponse
Input: session_id, question
Output: response (String)
<pre> 1: Begin 2: db.sessions.insert_one({ _id: session_id, user_id: user_id, type: interview_type, questions: questions, responses: [], start_time: current_timestamp() }) 3: wait until db.frontend_queue.has({ session_id, question }) 4: extract reply from queue 5: return reply 6: End </pre>

Algorithm 3: recordResponse
Input: session_id (String), question (String), response (String)
Output: response (String)
<pre> 1: Begin 2: db.sessions.update_one(3: { _id: session_id }, 4: { \$push: { responses: { question: question, answer: response, timestamp: current_timestamp() } } } 5:) 6: End </pre>

6.1.4 Algorithm: Answer Evaluation

Table 6.1.4 Answer Evaluation

Algorithm 1: Answer Evaluation
Input: session_id
Output: report_id (string)
<pre> Input: session_id Output: report_id 1: Begin 2: session ← db.sessions.find_one({ _id: session_id }) 3: (v_scores, nv_scores) ← evaluate_responses(session.responses) 4: report ← compile_report(session_id, v_scores, nv_scores) 5: result ← db.reports.insert_one(report) 6: return result.inserted_id 7: End </pre>

Algorithm 2: evaluateSessionResponses
Input: responses (List of {question, answer, timestamp})
Output: verbal_scores (List), nonverbal_scores (List)
<pre> 1: Begin 2: verbal_scores ← [] 3: nonverbal_scores ← [] 4: for each resp in responses do 5: v_score ← evaluateVerbalResponse(resp.answer, resp.question) 6: nv_score ← evaluateNonverbalResponse(resp.session_id, resp.question) 7: verbal_scores.append(v_score) 8: nonverbal_scores.append(nv_score) 9: end for 10: return (verbal_scores, nonverbal_scores) 11: End </pre>

Algorithm 3: evaluateVerbalResponse**Input: answer (String), question (String)****Output: score (Float)**

```

1: Begin
2: prompt ← buildVerbalEvalPrompt(question, answer)
3: return callLLM("answer-evaluator", prompt, max_tokens=200)
4: End

```

Algorithm 4: evaluateNonverbalResponse**Input: session_id (String), question (String)****Output: metrics (Object with rate, fillers, tone, pitch)**

```

1: Begin
2: audio ← fetchAudioSegment(session_id, question)
3: rate ← computeSpeechRate(audio)
4: fillers ← countFillerWords(audio)
5: tone ← analyzeTone(audio)
6: pitch ← analyzePitch(audio)
7: return { rate: rate, fillers: fillers, tone: tone, pitch: pitch }
8: End

```

Algorithm 5: compilePerformanceReport**Input: session_id, verbal_scores, nonverbal_scores****Output: report (Object)**

```

1: Begin
2: avg_verbal ← average(verbal_scores)
3: agg_nonverbal ← aggregateMetrics(nonverbal_scores)
4: feedback ← generateFeedback(verbal_scores, nonverbal_scores)
5: return {
6:   session_id: session_id,
7:   verbal_score: avg_verbal,
8:   nonverbal_score: agg_nonverbal,
9:   feedback: feedback,
10:  generated_at: current_timestamp()
11: }
12: End

```

6.1.5 Algorithm: Resume-Based Interview

Table 6.1.5 Resume-Based Interview

Algorithm 1: user_id, resume_file
Input: user_id, resume_file
Output: session_id (String)
<pre> 1: Begin 2: parsed ← parseResumeFile(resume_file) 3: db.resumes.insert_one({ user_id: user_id, parsed_data: parsed, uploaded_at: current_timestamp() }) 4: topics ← extractInterviewTopics(parsed) 5: session_id ← initiateResumeInterview(user_id, topics) 6: Return session_id 7: End </pre>

Algorithm 2: parseResumeFile
Input: file
Output: parsed_data (Object)
<pre> 1: Begin 2: text ← performOCR(file) 3: return nlpParseResume(text) 4: End </pre>

Algorithm 3: extractInterviewTopics
Input: parsed_data (Object)
Output: topics (List)
<pre> 1: Begin 2: return parsed_data.skills ∪ parsed_data.experience_topics 3: End </pre>

6.1.6 Algorithm: Chatbot Query Handling

Table 6.1.1 Chatbot Query Handling

Algorithm 1: Chatbot Query Handling
Input: user_id, chat_type, user_query
Output: chatbot_response
<pre> 1: Begin 2: if chat_type == "report" then 3: report ← db.reports.find_one({ user_id }, sort: { generated_at: -1 }) 4: context ← report.feedback 5: else if chat_type == "resume" then 6: resume ← db.resumes.find_one({ user_id }, sort: { uploaded_at: -1 }) 7: context ← resume.parsed_data 8: end if 9: prompt ← build_chatbot_prompt(context, user_query) 10: response ← call_llm("chatbot-model", prompt, max_tokens=300) 11: db.chatlogs.insert_one({ </pre>

```

    user_id, chat_type, question: user_query, answer: response, timestamp: now()
  })
12: return response
13: End

```

Algorithm 2: loadChatContext**Input: user_id (String), chat_type (String)****Output: context (Object)**

```

1: Begin
2: if chat_type == "report":
3:   rpt ← db.reports.find_one({ user_id: user_id }, sort={ generated_at: -1 })
4:   return rpt.feedback
5: else:
6:   res ← db.resumes.find_one({ user_id: user_id }, sort={ uploaded_at: -1 })
7:   return res.parsed_data
8: End

```

6.1.7 Algorithm: Progress Tracking Update**Table 6.1.1 Progress Tracking Update****Algorithm 1: Progress Tracking Update****Input: user_id, report_id****Output: progress_id (String)**

```

1: Begin
2: report ← db.reports.find_one({ _id: report_id })
3: record ← db.progress.find_one({ user_id: user_id })
4: (verbal_trend, nonverbal_trend) ← updateTrendData(record, report)
5: if record exists then
6:   db.progress.update_one({ user_id: user_id }, { $set: { trend_data: { verbal: verbal_trend,
nonverbal: nonverbal_trend }, last_updated: current_timestamp() }, $push: { sessions:
report.session_id } })
7:   Return record._id
8: else
9:   new_id ← generateUUID()
10:  db.progress.insert_one({ _id: new_id, user_id: user_id, sessions: [ report.session_id ], trend_data:
{ verbal: verbal_trend, nonverbal: nonverbal_trend }, last_updated: current_timestamp() })
11:  Return new_id
12: end if
13: End

```

Algorithm 2: updateTrendData**Input: record (Object|None), report (Object)****Output: (verbal_trend: List, nonverbal_trend: List)**

```

1: Begin
2: verbal_trend ← record?.trend_data.verbal or []
3: nonverbal_trend ← record?.trend_data.nonverbal or []
4: verbal_trend.append(report.verbal_score)
5: nonverbal_trend.append(report.nonverbal_score)
6: return (verbal_trend, nonverbal_trend)
7: End

```

Algorithm 3: calculatePerformanceImprovement**Input: trend_data (Object with verbal & nonverbal score lists)****Output: improvement_summary (Object with change metrics)**

```

1: Begin
2: if length(trend_data.verbal) < 2 or length(trend_data.nonverbal) < 2 then
3:   return { verbal_change: 0, nonverbal_change: 0 }
4: end if
5: v_change ← trend_data.verbal[-1] - trend_data.verbal[0]
6: nv_change ← trend_data.nonverbal[-1] - trend_data.nonverbal[0]
7: return { verbal_change: v_change, nonverbal_change: nv_change }
8: End

```

Algorithm 4: generateProgressGraphData**Input: user_id (String), trend_data (Object)****Output: graph_data (Object with labels and values)**

```

1: Begin
2: sessions ← db.sessions.find({ user_id: user_id }, sort={ start_time: 1 })
3: labels ← [ s.start_time.toString() for s in sessions ]
4: graph_data ← {
5:   labels: labels,
6:   verbal_scores: trend_data.verbal,
7:   nonverbal_scores: trend_data.nonverbal
8: }
9: return graph_data
10: End

```

6.2 External APIs/SDKs

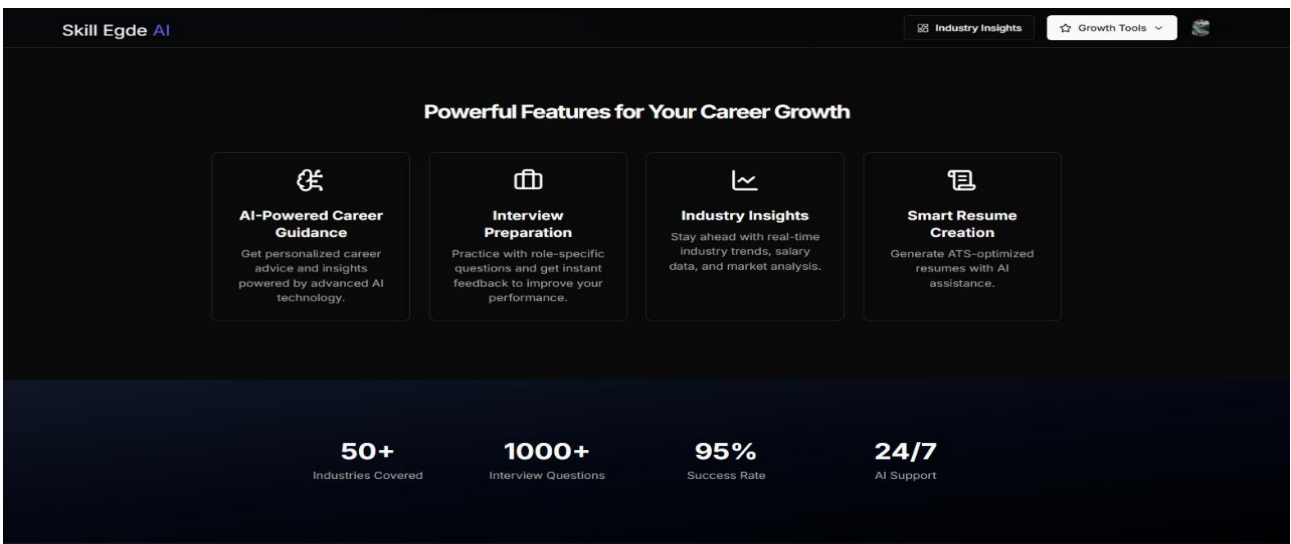
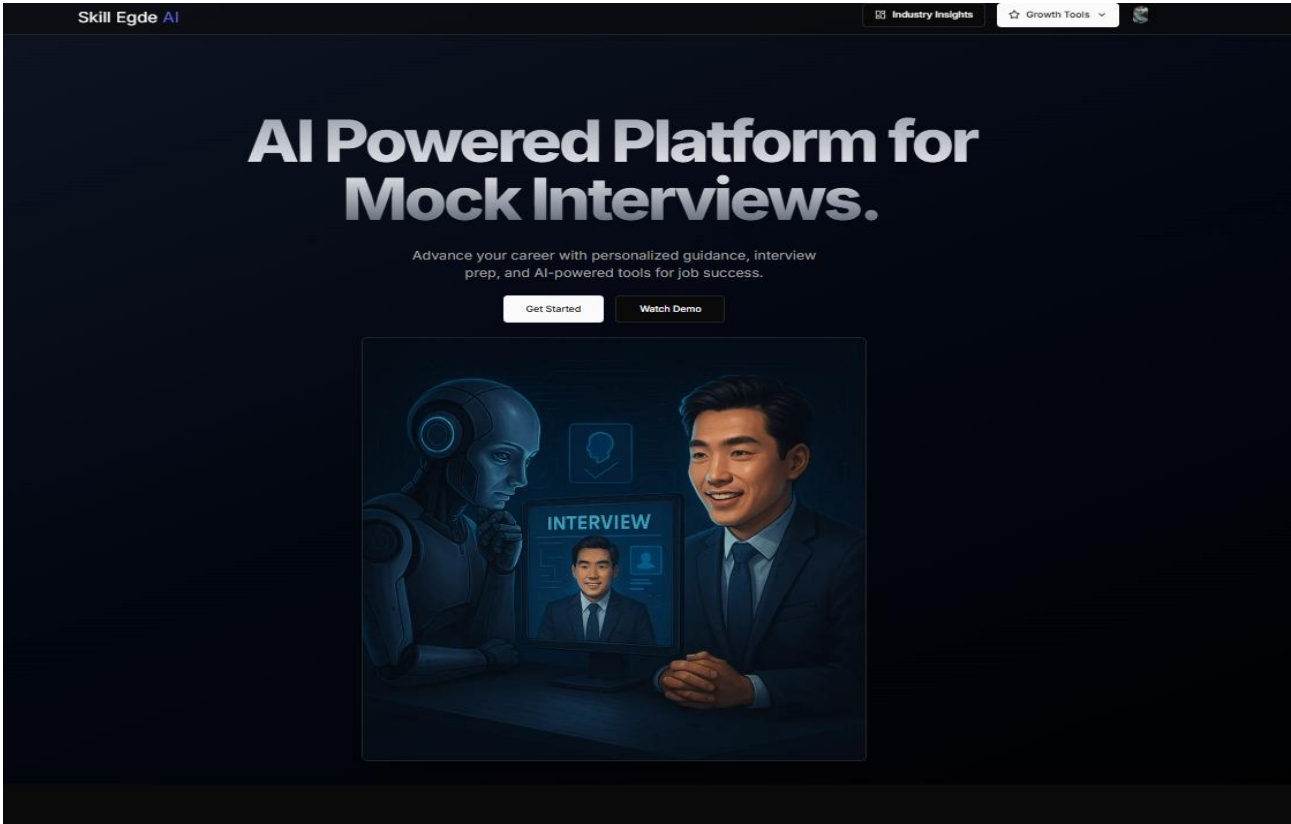
Following Table shows the APIs which will be used in SkillEdge AI.

Table 6.2 Details of APIs used in the project

Name of API and version	Description of API	Purpose of usage	Endpoint/Function/Class Used
<i>Clerk (Latest)</i>	A drop-in authentication and user management solution tailored for Next.js App Router.	Handle user sign-up, sign-in, session management, and protect both pages and API routes.	ClerkProvider (React component) • useAuth() hook • auth() helper
<i>Google Gemma-3 LLM</i>	State-of-the-art large-language model for text generation and completion tasks.	• Generate interview questions • Evaluate textual answer content and relevance	Gemma3.ChatCompletion.create() Gemma3.Completion.create()
<i>OpenAI Whisper API (v1)</i>	Automatic speech recognition model offering high-accuracy transcription.	Transcribe user audio answers into text for downstream verbal and non-verbal analysis	openai.Audio.transcriptions.create()
<i>FastAPI (v0.95.x)</i>	Python web framework for building APIs	Serve all backend microservices—question generation, evaluation, audio analysis, chatbot, etc.	@app.get(), @app.post(), dependency-injection via Depends
<i>Next.js Middleware</i>	Server-side route protection and session handling within Next.js.	Integrate Clerk's auth() and custom middleware to secure API routes and page access	middleware.js + withClerkMiddleware
<i>Supabase (Latest)</i>	Managed Postgres & realtime APIs	Store and retrieve structured data (users, sessions, reports, progress).	JavaScript client: supabase.from('table').select(), Python SDK methods

6.3 User Interface

6.3.1 Home Page



6.3.2 Profile Page

Skill Egde AI

Industry InsightsGrowth Tools

Enter Profile Details

Complete your profile to get started with SkillEdge

First Name

Last Name

Skills (comma separated)

e.g., JavaScript, React, Node.js

Industry

Position

Years of Experience

e.g., 3

Professional Bio

Tell us about your professional background...

Resume (PDF)

Choose file No file chosen

Upload Resume

Update Profile

Skill Egde AI

Industry InsightsGrowth Tools

Your Profile

Update Profile

Personal Information

First Name

Last Name

Farrukh

Zubair

Professional Details

Industry

Technology

Position

ML Engineer

Years of Experience

2 years

Skills

Next

Machine learning

Professional Bio

I am Farrukh Zubair .I am student of Comsats Universty Islamabad.

Resume

You have uploaded your resume.

6.3.3 Select Interview Screen

The screenshot shows the 'Select Interview Type' screen in the Skill Edge AI application. The interface has a dark blue background. At the top, there is a navigation bar with 'Skill Edge AI' on the left, 'Industry Insights' in the center, and 'Growth Tools' on the right. The main heading is 'Select Interview Type'. Below this, there are three selectable options: 'Technical' (which is selected and highlighted with a blue border), 'Behavioral', and 'Resume-Based'. Each option has a brief description: 'Technical' involves coding, algorithms, and system design; 'Behavioral' uses the STAR method for soft skills; 'Resume-Based' provides personalized questions from resume content. Below these options, there is a section 'Select a Technical Role:' with four buttons: 'Software Engineer' (selected), 'ML Engineer', 'Web Developer', and 'Mobile Developer'. A 'Continue' button is at the bottom center. A small 'N' icon is in the bottom left corner.

6.3.4 Interview Simulation Screen

The screenshot shows the 'Interview Simulation Screen' in the Skill Edge AI application. The interface has a dark blue background. At the top, there is a navigation bar with 'Skill Edge AI' on the left, 'Industry Insights' in the center, and 'Growth Tools' on the right. The main heading is 'Interview of ML Engineer'. On the left side, there is a circular avatar of a man in a suit and a video feed of a man in a cap. The main area contains a 'Question 1' box, a 'Your Answer' box with a blue circular progress indicator, and a 'Generating Questions...' box. Below the 'Your Answer' box, there is a text input field with the placeholder 'Type your answer here...'. At the bottom of the 'Your Answer' box, there are three buttons: 'Start Speaking', 'Stop Speaking', and 'Submit & Next'. A 'Time Left: 3:00' timer is in the top right corner. A 'Terminate Interview' button is in the bottom right corner. A small 'N' icon is in the bottom left corner.

Skill Edge AI

Industry Insights

Growth Tools

Interview of ML Engineer



Question 1

What are common metrics used to evaluate a model's performance on a given task?

Your Answer

Time Left: 2:12

Common metrics to evaluate model's performance are precision

Continue Speaking

Stop Speaking

Submit & Next

N

Terminate Interview

6.3.5 Interview Review Screen

Skill Edge AI

Industry Insights

Growth Tools

Your Interview Review

Question 1

What are common metrics used to evaluate a model's performance on a given task?

Your Answer:

Common metrics to evaluate model's performance are precision

Question 2

How do you handle imbalanced datasets in classification problems?

Your Answer:

Question 3

Describe a scenario where an ensemble method (like bagging or boosting) might outperform a single model.

Your Answer:

Question 4

How do you monitor model performance in production?

Your Answer:

7. Testing and Evaluation

All necessary testing and their test cases for SkillEdge AI are given below:

7.1 Unit Testing:

The Unit Tests of SkillEdge AI are given below their purpose is to test that whether it performs as designed.

7.1.1 Unit Testing 1: User Registration

Testing Objective: Ensure the sign-up form correctly handles valid and invalid inputs.

Table 7.1.1: Unit Test Cases for User Registration

No	Test Case/Test Script	Attribute/Values	Expected Result	Result
1	Validate email field for proper format	Email: invalidEmail	Highlights the email field and displays "Invalid email format" error message	Pass
2	Validate email field for required input	Email: ""	Highlights the email field and displays "Email is required" error message	Pass
3	Validate password field for required input	Password: ""	Highlights the password field and displays "Password is required" error message	Pass
4	Validate password strength	Password: abc	Highlights the password field and displays "Password must have 8 chars, 1 number, 1 special char"	Pass
5	Validate if Password Is Breached or not.	Password: 12345678	Highlights the breach password warning field and displays "This password has been found as part of a breach and can not be used"	Pass
6	Successful form submission with valid inputs	Email: raeeszakir34@gmail.com, Password: rz0139666	Redirects to OTP verification page	Pass

7.1.2 Unit Testing 2: User Login

Testing Objective: To ensure the login form is working correctly with valid and invalid credentials/inputs.

Table 7.1.2: Unit Test Cases for User Login

No	Test Case/Test Script	Attribute/Values	Expected Result	Result
1	Validate email field for required input	Email: ""	Displays "Email is required" error	Pass
2	Validate email format	Email: invalidEmail.com	Displays "Invalid email format" error	Pass
3	Validate password field for required input	Password: ""	Displays "Password is required" error	Pass
4	Validate login with invalid email/password	Email: wrong@domain.com, Password: wrongPass	Displays "Invalid credentials" error	Pass
5	Validate login with correct credentials	Email: raeeszakir34@gmail.com, Password: rz0139666	Redirects to dashboard	Pass

7.1.3 Unit Testing 3: Update Profile Page

Testing Objective: To ensure User enters correct and relevant information in the profile page.

Table 7.1.3: Unit Test Cases for User Profile Page

No.	Test Case / Test Script	Attribute / Values	Expected Result	Result
1	Validate "First Name" required	firstName: ""	Highlights First Name field and displays "First name is required"	Pass
2	Validate "Last Name" required	lastName: ""	Highlights Last Name field and displays "Last name is required"	Pass
3	Validate "Skills" input splits and trims	skills: " JavaScript , React , "	formData.skills parsed to ["JavaScript","React"]	Pass
4	Validate "Industry" selection required	industry: ""	Highlights Industry dropdown and displays "Industry is required"	Pass
5	Validate "Position" selection required	position: ""	Highlights Position dropdown and displays "Position is required"	Pass
6	Validate "Experience" numeric and non-negative	experience: "-3"	Highlights Experience field and displays "Experience must be ≥ 0 "	Pass
7	Validate "Bio" max-length (e.g. 500 chars)	bio: "<501 chars>"	Highlights Bio field and displays "Bio must be 500 characters or less"	Pass
8	Successful form	firstName: "Raees",	PUT /api/profile returns 200 OK,	Pass

	submission with valid data	lastName: "Zakir", skills: "JS,React", industry: "Technology", position: "Software Engineer", experience: "3", bio: "..."	toast success, redirect to /profile	
9	Validate resume upload rejects non-PDF files	resumeFile: { name: "doc.txt", type: "text/plain" }	Displays “Please select a valid PDF file.”	Pass
10	Validate resume upload success flow	resumeFile: valid PDF blob	Upload button shows “Uploading...”, then “Resume uploaded and linked to your profile!” message appears	Pass
11	Pre-fill form when no profile exists	/api/profile returns 404	First/Last Name pre-filled from Clerk user.firstName/user.lastName, other fields remain empty	Pass
12	Handle profile fetch network error	simulate fetch /api/profile network failure	Displays error container with “Network error. Please try again later.” and toast error	Pass

7.1.4 Unit Testing 4: Question Generation & Session Management

Testing Objective: To ensure questions are generated in correct format by the model.

Table 7.1.4: Unit Test Cases for Question Generation & Session Management

No.	Test Case / Test Script	Attribute / Values	Expected Result	Result
1	generate_questions with valid interview type	user_id: valid UUID type: "technical"	HTTP 200; returns non-empty JSON array of question strings	Pass
2	generate_questions with unsupported interview type	user_id: valid UUID type: "unknown"	HTTP 400; JSON error { "detail": "Unsupported interview type" }	Pass
3	init_session stores session metadata	session_id: new UUID user_id: valid UUID type: "behavioral"	Supabase sessions table contains a row with those fields and a non-null start_time, empty responses array	Pass
4	record_response appends to session responses array	session_id: existing UUID question: "What is AI?" response: "Artificial Intelligence"	That session's responses array length increments by 1 and includes { question: "What is AI?", answer: "Artificial Intelligence", timestamp: ... }	Pass

5	close_session sets end_time	session_id: existing UUID	Supabase sessions row for that session_id has a non-null end_time equal to the time of the call	Pass
6	Full simulation flow returns a session ID	POST /api/sessions with valid user_id and type: "technical"	HTTP 201; JSON { "session_id": "<UUID>" }; new session record exists with start_time, no end_time until closed	Pass

7.2 Functional Testing:

The purpose of functional testing is to verify that each module of the SkillEdge AI system performs as specified in the functional requirements. Following are the sub-section corresponding to a major module:

7.2.1 Functional Testing 1: Interview Simulation Flow

Table 7.2.1: Functional Testing 1: Interview Simulation Flow

NO.	Test Case / Test Script	Input	Expected Result	Result
1	Start a new interview session	POST /api/sessions { user_id: validUUID, type: "technical" }	201 Created, JSON { session_id: "<UUID>" }	Pass
2	Full Q&A loop	Use returned session_id to:1. GET:/api/sessions/{id}/next-question2. POST:/api/sessions/{id}/responses repeatedly	Each question endpoint returns a question string; each response is recorded; after last, session closes (end_time set)	Pass
3	await_frontend response timeout	Do not POST any responses within timeout period.	After timeout, GET /api/sessions/{id}/status returns {"status":"timeout","message":"No response received"}, HTTP 504	Pass
4	Start session with unsupported interview type	POST /api/sessions { user_id: validUUID, type: "foo" }	400 Bad Request, JSON { "detail": "Unsupported interview type" }	Pass

5	Verify session metadata persisted correctly	After successful POST, GET /api/sessions/{session_id}	2+A1:D600 OK; JSON includes: { session_id, user_id, type, questions: [...], start_time, end_time: null }	Pass
---	---	--	--	------

7.2.2 Functional Testing 2: Resume-Based Interview

Table 7.2.2: Functional Testing 2: Resume-Based Interview

NO.	Test Case / Test Script	Input	Expected Result	Result
FT3.1	Upload valid résumé	POST /api/resume with form-data field file = valid PDF	200 OK; JSON body { resume_id: "<UUID>", parsed_data: { skills: [...], experience: [...], education: [...] } }	
FT3.2	Upload unsupported file format	POST /api/resume with file = "malware.exe"	415 Unsupported Media Type; JSON { detail: "Only PDF resumes are supported" }	
FT3.3	Upload empty request	POST /api/resume with no file	400 Bad Request; JSON { detail: "Resume file is required" }	
FT3.4	Start résumé-based session	POST /api/sessions with JSON { user_id: validUUID, resume_id: "<validResumeId>" }	201 Created; JSON { session_id: "<UUID>" }; new session record has type: "resume" and questions array populated based on parsed resume topics	
FT3.5	Start résumé-based session with invalid resume_id	POST /api/sessions with { user_id: validUUID, resume_id: "nonexistent" }	404 Not Found; JSON { detail: "Resume not found" }	

7.3 Integration Testing:

7.3.1 Integration Testing 1: User Management

Integration Testing 1: User Registration (Signup)

Testing Objective: To ensure the user registration flow works correctly, validating email uniqueness, password format, and integration with OTP verification.

Table 7.3.1: Integration Testing for User Management

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Signup with valid details	Name: Raees, Email: raees@gmail.com, Password: raees@123	Redirect to OTP verification screen	Redirected to OTP	Pass
2	Signup with existing email	Email: raees@gmail.com	Error message: "Account already exists"	Error displayed	Pass
3	Signup with invalid email format	Email: raees@com	Error message: "Invalid email format"	Error displayed	Pass
4	Signup with weak password	Password: 123456	Error message: "Password must be strong"	Error displayed	Pass
5	OTP verification after signup	OTP: 645321	Redirect to login screen	Redirected to login	Pass
6	OTP not entered or incorrect	OTP: (empty or incorrect)	Error message: "Invalid or missing OTP"	Error displayed	Pass

7.3.2 Integration Testing 2: User Registration (Login)

Testing Objective: To verify login with valid and invalid credentials, and ensure role-based redirection to the correct dashboard.

Table 7.3.2: Integration Testing for Login

No.	Test Case/Test Script	Attribute and Value	Expected Result	Actual Result	Result
1	Login with valid student credentials	Email: student01@gmail.com, Password: Student@123	Redirect to student dashboard	Redirected to dashboard	Pass
3	Login with incorrect password	Email: student01@gmail.com, Password: wrongPass	Error message: "Invalid credentials"	Error displayed	Pass
4	Login with unregistered email	Email: unknown@gmail.com, Password: AnyPassword123	Error message: "Account not found"	Error displayed	Pass
5	Login with empty fields	Email: (empty), Password: (empty)	Error message: "Fields cannot be empty"	Error displayed	Pass

7.3.3 Integration Testing 3: Interview Simulator

Testing Objective: To verify model generated questions and user can see the questions at front end and answer them.

Table 7.3.3: Integration Testing for Interview Simulator

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Start a new interview session	POST /api/sessions• user_id: valid UUID• type: “technical”	HTTP 201, JSON { session_id: " <u><UUID></u> " } Session record exists with start_time, type=“technical”, non-empty questions[], end_time=null	Pass
2	Retrieve first question	GET /api/sessions/{session_id}/next• session_id from Test 1	HTTP 200, JSON { question: " <u><first question text></u> " }	Pass
3	Loop through all questions and submit answers	For each returned question:1. POST /api/sessions/{session_id}/responses• { question: q, answer: "test" }	Each POST → HTTP 200After last answer, GET /api/sessions/{session_id}/next returns { status: "finished" }	Pass
4	Verify session closure and response storage	1. Complete Test 32. GET /api/sessions/{session_id}	HTTP 200, JSON includes:• end_time ≠ null• responses.length = questions.length• responses match submitted answers	Pass
5	Invalid session handling	1. GET /api/sessions/invalid-id/next2. POST /api/sessions/invalid-id/responses	Both → HTTP 404, JSON { detail: "Session not found" }No new records created in DB	Pass

Document Viewer

Turnitin Originality Report

Processed on: 26-May-2025 3:38 AM PST

ID: 2601204565

Word Count: 3305

Submitted: 1

Report By Raees Zakir .

Similarity Index		Similarity by Source	
20%		Internet Sources:	12%
		Publications:	1%
		Student Papers:	16%
include quoted	include bibliography	exclude small matches	mode:
quickview (classic) report	print	refresh	download

10% match (Internet from 26-Apr-2023)

<https://www.coursehero.com/file/p17vnm8/Example-PER-1-95-of-webpages-generated-by-the-COS-shall-download-completely/> ✕

5% match (Internet from 16-May-2024)

<https://www.coursehero.com/file/p14aud4g/Table-1-Description-of-FR-1-Identifier-FR-1-6-Title-Title-of-requirement/> ✕

3% match (Internet from 16-Sep-2018)

<http://wdz.eng.br> ✕

2% match (Internet from 04-Apr-2024)

<http://riogrande.cs.tcu.edu> ✕

1% match (Internet from 11-Jan-2023)

<https://pdfcoffee.com/srs-emotion-detection-pdf-free.html> ✕

1% match (student papers from 27-Jun-2024)

[Submitted to University of Maryland, Global Campus on 2024-06-27](#) ✕

1% match (student papers from 15-Nov-2023)

[Submitted to University of Houston Clear Lake on 2023-11-15](#) ✕

<1% match (student papers from 13-Sep-2018)

[Submitted to University of Houston Clear Lake on 2023-09-13](#) ✕